cna: An R Package for Configurational Causal Inference and Modeling

Michael Baumgartner
University of Bergen, Norway

Mathias Ambühl
Consult AG, Switzerland

Abstract

The R package **cna** provides comprehensive functionalities for causal inference and modeling with *Coincidence Analysis* (CNA), which is a configurational comparative method of causal data analysis. In this vignette, we first review the theoretical and methodological background of CNA. Second, we introduce the data types processable by CNA, the package's core analytical functions with their arguments, and some auxiliary functions for data simulations. Third, CNA's output along with relevant fit parameters and output attributes are discussed. Fourth, we provide guidance on how to interpret that output and, in particular, on how to proceed in case of model ambiguities. Finally, some considerations are offered on benchmarking the reliability of CNA.

Keywords: configurational comparative methods, set-theoretic methods, Coincidence Analysis, Qualitative Comparative Analysis, INUS causation, Boolean causation.

1. Introduction

Coincidence Analysis (CNA) is a configurational comparative method of causal data analysis that was introduced for crisp-set (i.e. binary) data in (Baumgartner 2009a; 2009b; 2015), generalized for multi-value and fuzzy-set data in (Baumgartner and Ambühl 2020), and complemented by a robustness protocol in (Parkkinen and Baumgartner 2023). In recent years, CNA has been applied in numerous studies across the social, political, and behavioral sciences, with a particularly rapid uptick in usage in public health, covering a wide range of topics such as colorectal cancer screening, patient safety in nursing homes, implementation of Hepatitis C virus treatments, drug withdrawal, COVID-19 vaccination rates, or the connection between firearm laws and homicide rates.[1] In contrast to more standard methods of data analysis, which primarily quantify effect sizes, CNA belongs to a family of methods designed to group causal influence factors conjunctively (i.e. in complex bundles) and disjunctively (i.e. on alternative pathways). It is firmly rooted in a so-called regularity theory of causation and it is the only method of its kind that can recover causal structures with multiple outcomes (effects), for example, causal chains.

Many disciplines investigate causal structures with one or both of the following features: (i) causes are arranged in complex bundles that only become operative when all of their components are properly co-instantiated, each of which in isolation is ineffective or leads to

[1] The Zotero CNA library provides detailed references to more than 180 applications of CNA. Among other impacts, CNA has been showcased in the flagship journal of implementation science (Whitaker *et al.* 2020).

#	S	C	F
c_1	1	1	1
c_2	0	0	1
c_3	1	0	0
c_4	0	1	0

(a)

	S	C	F
S	1.00	0.00	0.00
C	0.00	1.00	0.00
F	0.00	0.00	1.00

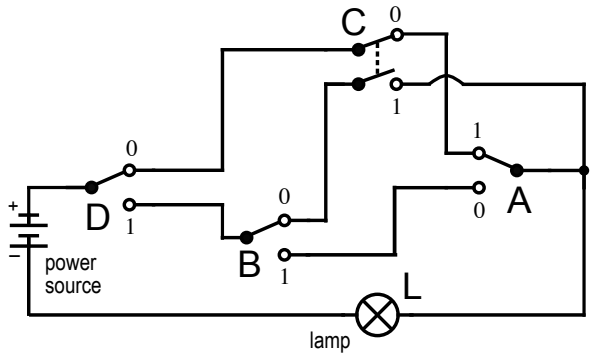
(b)

Table 1: Table (a) contains ideal configurational data, where each row depicts a different configuration of the factors S, C and F. Configuration c_1, for example, represents cases (units of observation) in which all factors take the value 1, whereas in c_2, S and C are 0 and F is 1, etc. Table (b) is the corresponding correlation matrix.

different outcomes, and (ii) outcomes can be brought about along alternative causal routes such that, when one route is suppressed, the outcome may still be produced via another one. For example, from a given set of implementation strategies available to medical facilities, some strategies yield a desired outcome (e.g. high uptake of treatment innovation) in combination with certain other strategies, whereas in different combinations the same strategies may have opposite or no effects (e.g. Yakovchenko *et al.* 2020). Or, a variation in a phenotype only occurs if many single-nucleotide polymorphisms interact, and various such interactions can independently induce the same phenotype (e.g. Culverhouse *et al.* 2002). Different labels are used for features (i) and (ii) in different disciplines: "interactions", "component causation", "conjunctural causation", "alternative causation", "equifinality", etc. For uniformity's sake, we will subsequently refer to (i) as *conjunctivity* and to (ii) as *disjunctivity* of causation, reflecting the fact that causes form conjunctions and disjunctions, that is, Boolean AND- and OR-connections.

Causal structures featuring conjunctivity and disjunctivity pose severe challenges for methods of causal data analysis. As many theories of causation entail that it is necessary (though not sufficient) for X to be a cause of Y that there be some kind of dependence (e.g. probabilistic or counterfactual) between X and Y, standard methods—for instance, regression and Bayesian network methods—infer that X is *not* a cause of Y if X and Y are *not* pairwise dependent (i.e. correlated). However, structures displaying conjunctivity and disjunctivity often do not exhibit pairwise dependencies. As a very simple illustration, consider the interplay between a person's skills to perform an activity, the challenges posed by that activity, and the actor's autotelic experience of complete involvement with the activity called *flow* (Csikszentmihalyi 1975). A binary model of this interplay involves the factors S, with values 0/1 representing low/high skills, C, with 0/1 standing for low/high challenges, and F, with 0/1 representing the absence/presence of flow. Csikszentmihalyi's (1975, ch. 4) flow theory entails that flow is triggered if, and only if, skills and challenges are either both high or both low, meaning that F=1 has the two alternative causes S=1 & C=1 and S=0 & C=0. If the flow theory is true, ideal (i.e. non-fragmented, unconfounded, noise-free) data on this structure feature the four configurations c_1 to c_4 in Table 1a, and no others. As can easily be seen from the corresponding correlation matrix in Table 1b, there are no pairwise dependencies. In consequence, standard methods will struggle to find the flow model, even when processing ideal data on it.

Although standard methods provide various protocols for tracing interaction effects involving two or three exogenous factors, these interaction calculations face tight computational



(a)

	A	*B*	*C*	*D*	*L*
A	1.00	0.00	0.00	0.00	0.00
B	0.00	1.00	0.00	0.00	0.00
C	0.00	0.00	1.00	0.00	0.00
D	0.00	0.00	0.00	1.00	0.26
L	0.00	0.00	0.00	0.26	1.00

(b)

	A	*B*	*D*	*C*	*L*
c_1	0	1	1	1	1
c_2	1	0	1	1	1
c_3	0	0	1	1	1
c_4	0	1	1	0	1
c_5	1	1	0	0	1
c_6	1	0	0	0	1
c_7	1	1	1	1	0
c_8	1	1	0	1	0
c_9	0	1	0	1	0
c_{10}	1	0	0	1	0
c_{11}	0	0	0	1	0
c_{12}	1	1	1	0	0
c_{13}	1	0	1	0	0
c_{14}	0	0	1	0	0
c_{15}	0	1	0	0	0
c_{16}	0	0	0	0	0

(c)

Figure/Table 2: Diagram (a) depicts a simple electrical circuit with three single-pole switches D, B, A, one double-pole switch C, and one lamp L. Table (c) comprises ideal data on that circuit and Table (b) the correlation matrix corresponding to that data.

complexity restrictions when more exogenous factors are involved and quickly run into multicollinearity issues (Brambor *et al.* 2006). Yet, structures with conjunctivity and disjunctivity may be much more complex than the flow model. Consider the electrical circuit in Figure 2a. It comprises a lamp L that can be on or off and four switches A to D, each of which can either be in position 1 or position 0. There are three alternative conjunctions of switch positions that close the circuit and cause the lamp to be on: A=0 & B=1 & D=1 OR A=1 & C=0 & D=0 OR B=0 & C=1 & D=1. As the switches are mutually independent, there are $2^4 = 16$ logically possible configurations of switch positions. For each of these configurations c_1 to c_{16}, Table 2c lists whether the lamp is on (L=1) or off (L=0). That table thus contains all and only the empirically possible configurations of the five binary factors representing the switches and the lamp. These are ideal data for the circuit in Figure 2a. Yet, even though all of the switch positions are causes of the lamp being on in some combination or other, factors A, B, and C are pairwise *independent* of L; only D is weakly correlated with L, as can be seen from the correlation matrix in Table 2b (which results from Table 2c). Standard methods of causal data analysis cannot infer the causal structure behind that circuit from Table 2c. They are not designed to group causes conjunctively and disjunctively.

A switch position as A=0 can only be identified as cause of L=1 by finding the whole conjunction of switch positions in which A=0 is indispensable for closing the circuit. More generally, discovering causal structures exhibiting conjunctivity and disjunctivity calls for a method that tracks causation as defined by a theory not treating a dependence between individual causes and effects as necessary for causation and that embeds values of exogenous factors in complex Boolean AND- and OR-functions over many other causes, fitting those functions as a whole to the data. But the space of Boolean functions over even a handful of factors is vast. For n binary factors there exist 2^{2^n} Boolean functions. For the switch positions in our circuit there exist 65536 Boolean functions; if we add only one additional binary switch that

number jumps to 4.3 billion, and if we also consider factors with more than two values, that number becomes computationally unmanageable. That means a method capable of correctly discovering causal structures with conjunctivity and disjunctivity must find ways to efficiently navigate in that vast space of possibilities. This is the purpose of CNA.

CNA takes data on binary, multi-value or fuzzy-set factors as input and infers causal structures as defined by the so-called *INUS theory* from it. The INUS theory was first developed by Mackie (1974) and later refined to the *MINUS theory* by Graßhoff and May (2001) and Baumgartner and Falk (2023a) (see also Beirlaen *et al.* 2018 and Zhang and Zhang 2023). It defines causation in terms of redundancy-free Boolean dependency structures and, importantly, does not require causes and their outcomes to be pairwise dependent. As such, it is custom-built to account for structures featuring conjunctivity and disjunctivity.

CNA is not the only method for the discovery of (M)INUS structures. Other methods that can be used for that purpose are Logic Regression (Ruczinski *et al.* 2003; Kooperberg and Ruczinski 2005), which is implemented in the R package **LogicReg** (Kooperberg and Ruczinski 2023),[2] and Qualitative Comparative Analysis (QCA; Ragin 2008; Rihoux and Ragin 2009; Cronqvist and Berg-Schlosser 2009; Thiem 2018), implemented in the R package **QCA** (Dusa 2026).[3] But CNA is the only method of its kind that can build models with more than one outcome and, hence, can analyze common-cause and causal chain structures as well as causal cycles and feedbacks. Moreover, unlike the models produced by Logic Regression or Qualitative Comparative Analysis, CNA's models are guaranteed to be redundancy-free, which makes them directly causally interpretable in terms of the (M)INUS theory; and CNA is more successful than any other method at exhaustively uncovering *all* (M)INUS models that fit the data equally well. For detailed comparisons of CNA with Qualitative Comparative Analysis and Logic Regression see (Baumgartner and Ambühl 2020; Swiatczak 2021) and (Baumgartner and Falk 2023b), respectively.

The **cna** package reflects and implements CNA's latest stage of development. This vignette provides a detailed introduction to **cna**. We first exhibit **cna**'s theoretical and methodological background. Second, we discuss the main inputs of the package's core function `cna()` along with numerous auxiliary functions for data review and simulation. Third, the working of the algorithm implemented in `cna()` is presented. Fourth, we explain `cna()`'s output along with relevant fit parameters and output attributes. Fifth, we provide some guidance on how to interpret that output and, in particular, on how to proceed in case of model ambiguities. Finally, some considerations are offered on benchmarking the reliability of `cna()`.

2. Background

The (M)INUS theory of causation belongs to the family of so-called *regularity theories*, which have roots as far back as Hume (1999 (1748)). It is a type-level theory of causation (cf. Baumgartner 2020) that analyzes the dependence relation of causal relevance between factors/variables taking on specific values, as in "X=χ is causally relevant to Y=γ". It assumes that causation is ultimately a deterministic form of dependence, such that whenever the same complete cause occurs the same effect follows. This entails that indeterministic behavior

[2] Another package implementing a variation of Logic Regression is **logicFS** (Schwender and Tietz 2024).

[3] Other useful QCA software include **QCAfalsePositive** (Braumoeller 2015) and **SetMethods** (Oana *et al.* 2025).

patterns in data result from insufficient control over background influences generating noise and not from the indeterministic nature of the underlying causal processes. For X=χ to be a (M)INUS cause of Y=γ, X=χ must be a difference-maker of Y=γ, meaning—roughly—that there exists a context in which other causes take constant values and a change from X$\neq\chi$ to X=χ is associated with a change from Y$\neq\gamma$ to Y=γ.

To further clarify that theory as well as the characteristics and requirements of inferring (M)INUS structures from empirical data a number of preliminaries are needed.

2.1. Factors and their values

Factors are the basic modeling devices of CNA. They are analogous to (random) variables in statistics, that is, they are functions from (measured) properties into a range of values. They can be used to represent categorical properties that partition sets of units of observation (cases) either into two sets, in case of binary properties, or into more than two (but finitely many) sets, in case of multi-value properties, such that the resulting sets are exhaustive and pairwise disjoint. Factors representing binary properties can be *crisp-set* (*cs*) or *fuzzy-set* (fs); the former can take on 0 and 1 as possible values, whereas the latter can take on any (continuous) value from the unit interval $[0, 1]$. Factors representing multi-value properties are called *multi-value (mv) factors*; they can take on any of a finite (but not fixed) number of non-negative integers.

Values of a *cs* or fs factor X can be interpreted as membership scores in the set of cases exhibiting the property represented by X. A case of type X=1 is a full member of that set, a case of type X=0 is a (full) non-member, and a case of type X=χ_i, $0 < \chi_i < 1$, is a member to degree χ_i. A case is considered a member of X if its membership score χ_i reaches the 0.5-anchor, that is, $\chi_i \geq 0.5$, and it is a non-member of X if $\chi_i < 0.5$. An alternative interpretation, which lends itself particularly well for causal modeling, is that "X=1" stands for the full presence of the property represented by X, "X=0" for its full absence, and "X=χ_i" for its partial presence (to degree χ_i). By contrast, the values of an *mv* factor X designate the particular way in which the property represented by X is exemplified. For instance, if X represents the education of subjects, X=2 may stand for "high school", with X=1 ("no completed primary schooling") and X=3 ("university") designating other possible property exemplifications. *Mv* factors taking on one of their possible values also define sets, but the values themselves must not be interpreted as membership scores; rather they denote the relevant property exemplification.

As the explicit "Factor=value" notation yields convoluted syntactic expressions with increasing model complexity, the **cna** package uses the following shorthand notation, which is standard in Boolean algebra (Bowran 1965): membership in a set is expressed by italicized upper case and non-membership by italicized lower case letters. "X" signifies membership in the set of cases exhibiting the property represented by X and "x" signifies non-membership in that set. Italicization thus carries meaning: "X" designates the factor and "X" membership in the set of cases with values of X above 0.5. In case of *mv* factors, value assignments to factors are not abbreviated but always written out, using the "Factor=value" notation.

2.2. Boolean operations

The (M)INUS theory defines causation using the Boolean operations of negation ($\neg X$, or x),

Inputs		Outputs				
X	Y	$\neg X$	$X{*}Y$	$X+Y$	$X \rightarrow Y$	$X \leftrightarrow Y$
1	1	0	1	1	1	1
1	0	0	0	1	0	0
0	1	1	0	1	1	0
0	0	1	0	0	1	1

Table 2: Classical Boolean operations applied to *cs* factors.

conjunction ($X{*}Y$), disjunction ($X+Y$), implication ($X \rightarrow Y$), and equivalence ($X \leftrightarrow Y$).[4] Negation is a unary truth function, the other operations are binary truth functions. That is, they take one or two truth values as inputs and output a truth value. When applied to *cs* factors, both their input and output set is $\{0,1\}$. Negation is translated by "not", conjunction by "and", disjunction by "or", implication by "if ... then", and equivalence by "if and only if (iff)". Their classical definitions are given in Table 2.

These operations can be straightforwardly applied to *mv* factors as well, in which case they amount to functions from the *mv* factors' domain of values into the set $\{0,1\}$. To illustrate, let both X and Y be ternary factors with values from the domain $\{0,1,2\}$. The negation of X=2, *viz.* $\neg$(X=2), then returns 1 iff X is not 2, meaning iff X is 0 or 1. X=2*Y=0 yields 1 iff X is 2 and Y is 0. X=2 + Y=0 returns 1 iff X is 2 or Y is 0. X=2 $\rightarrow$ Y=0 yields 1 iff X is not 2 or Y is 0. X=2 $\leftrightarrow$ Y=0 issues 1 iff either X is 2 and Y is 0 or X is not 2 and Y is not 0.

For fs factors, the classical Boolean operations must be translated into fuzzy logic. There exist numerous systems of fuzzy logic (for an overview cf. Hájek 1998), many of which come with their own renderings of Boolean operations. In the context of CNA (and QCA), the following fuzzy-logic rendering is standard: negation $\neg X$ is $1-X$, conjunction $X{*}Y$ is the minimum membership score in X and Y, i.e., $\min(X,Y)$, disjunction $X+Y$ is the maximum membership score in X and Y, i.e., $\max(X,Y)$, an implication $X \rightarrow Y$ is taken to express that the membership score in X is smaller or equal to Y ($X \leq Y$), and an equivalence $X \leftrightarrow Y$ that the membership scores in X and Y are equal ($X = Y$).

Based on the implication operator, the notions of *sufficiency* and *necessity* are defined, which are the two Boolean dependencies exploited by the (M)INUS theory:

Sufficiency X is sufficient for Y iff $X \rightarrow Y$ (or equivalently: $x+Y$; and colloquially: "if X is present, then Y is present");

Necessity X is necessary for Y iff $Y \rightarrow X$ (or equivalently: $x \rightarrow y$ or $y+X$; and colloquially: "if Y is present, then X is present").

Analogously for more complex expressions:

- X=3*Z=2 is sufficient for Y=4 iff X=3*Z=2 $\rightarrow$ Y=4;
- X=3 + Z=2 is necessary for Y=4 iff Y=4 $\rightarrow$ X=3 + Z=2;
- X=3 + Z=2 is sufficient and necessary for Y=4 iff X=3 + Z=2 $\leftrightarrow$ Y=4.

[4] Note that "*" and "+" are used as in Boolean algebra here, which means, in particular, that they do not represent the linear algebraic (arithmetic) operations of multiplication and addition (notational variants of Boolean "*" and "+" are "$\wedge$" and "$\vee$"). For a standard introduction to Boolean algebra see (Bowran 1965).

2.3. (M)INUS causation

Boolean dependencies of sufficiency and necessity amount to mere patterns of co-occurrence of factor values; as such, they carry no causal connotations whatsoever. In fact, most Boolean dependencies do not reflect causal dependencies. To mention two well-rehearsed examples: the sinking of a properly functioning barometer in combination with high temperatures and blue skies is sufficient for weather changes, but it does not cause the weather; or whenever it rains, the street gets wet, hence, wetness of the street is necessary for rainfall but certainly not causally relevant for it. At the same time, some dependencies of sufficiency and necessity are in fact due to underlying causal dependencies: rainfall is sufficient for wet streets and also a cause thereof, or the presence of oxygen is necessary for fires and also a cause thereof.

That means the crucial problem to be solved by the (M)INUS theory is to filter out those Boolean dependencies that are due to underlying causal dependencies and are, hence, amenable to a causal interpretation. The main reason why most sufficiency and necessity relations do not reflect causation is that they either contain redundancies or are themselves redundant to account for the behavior of the outcome, whereas causal conditions do not feature redundant elements and are themselves indispensable to account for the outcome in at least one context. Accordingly, to filter out the causally interpretable Boolean dependencies, they need to be freed of redundancies. In Mackie's (1974, 62) words, causes are *I*nsufficient but *N*on-redundant parts of *U*nnecessary but *S*ufficient conditions (thus the acronym INUS).

While Mackie's INUS theory only requires that sufficient conditions be freed of redundancies, he himself formulates a problem for that theory, *viz.* the *Manchester Factory Hooters* problem (Mackie 1974, 81-87), which Graßhoff and May (2001) solve by eliminating redundancies also from necessary conditions. Accordingly, modern versions of the INUS theory stipulate that whatever can be removed from sufficient or necessary conditions without affecting their sufficiency and necessity is not a difference-maker and, hence, not a cause. The causally interesting sufficient and necessary conditions are *minimal* in the following sense:

Minimal sufficiency A conjunction Φ of coincidently instantiated factor values (e.g., $X_1 * \ldots * X_n$) is a minimally sufficient condition of Y iff $\Phi \rightarrow Y$ and there does not exist a proper part Φ' of Φ such that $\Phi' \rightarrow Y$, where a proper part Φ' of Φ is the result of eliminating one or more conjuncts from Φ.

Minimal necessity A disjunction Ψ of minimally sufficient conditions (e.g., $\Phi_1 + \ldots + \Phi_n$) is a minimally necessary condition of Y iff $Y \rightarrow \Psi$ and there does not exist a proper part Ψ' of Ψ such that $Y \rightarrow \Psi'$, where a proper part Ψ' of Ψ is the result of eliminating one or more disjuncts from Ψ.

Minimally sufficient and minimally necessary conditions can be combined to so-called *atomic MINUS-formulas* (Beirlaen *et al.* 2018; or, equivalently, *minimal theories*, Graßhoff and May 2001):

Atomic MINUS-formula An atomic MINUS-formula of an outcome Y is an expression $\Psi \leftrightarrow Y$, where Ψ is a minimally necessary disjunction of minimally sufficient conditions of Y, in disjunctive normal form (DNF).[5]

[5] An expression is in DNF iff it is a disjunction of one or more conjunctions of one or more literals (i.e. factor values; Lemmon 1965, 190; or Bowran 1965, 13).

Atomic MINUS-formulas can represent structures with one outcome only. To represent structures with more than one outcome, atomic MINUS-formulas are conjunctively combined to *complex MINUS-formulas*. But conjunctive concatenation can introduce new redundancies. It is possible that a conjunction of, say, three atomic MINUS-formulas $\mathbf{m}_1{*}\mathbf{m}_2{*}\mathbf{m}_3$ is logically equivalent to a conjunction of only two of them, for instance, $\mathbf{m}_1{*}\mathbf{m}_2$ (see Baumgartner and Falk 2023a for a concrete example). In that case, $\mathbf{m}_3$ makes no difference to the behavior of the factors in the structure, meaning it is redundant. Baumgartner and Falk (2023a) call this a *structural redundancy*. Consequently, the definition of a complex MINUS-formula must include an additional non-redundancy constraint:

Complex MINUS-formula A complex MINUS-formula of outcomes $Y_1, \ldots, Y_n$ is a conjunction $(\Psi_1 \leftrightarrow Y_1){*}\ldots{*}(\Psi_n \leftrightarrow Y_n)$ of atomic MINUS-formulas that is itself free of structural redundancies.

Both atomic and complex MINUS-formulas are referred to as *MINUS-formulas*, for short. They serve as a bridge between Boolean dependencies and causal dependencies: only those Boolean dependencies are causally interpretable that appear in MINUS-formulas. To make this concrete, consider the following atomic exemplar:

$$A{*}e + C{*}d \leftrightarrow B \tag{1}$$

(1) being a MINUS-formula of B entails that $A{*}e$ and $C{*}d$, but neither A, e, C, nor d alone, are sufficient for B and that $A{*}e + C{*}d$, but neither $A{*}e$ nor $C{*}d$ alone, are necessary for B. If this holds, it follows that for each (appearance of a) factor value in (1) there exists a *difference-making pair*, meaning a pair of configurations such that a change in that factor value alone accounts for a change in the outcome (Baumgartner and Falk 2023a). For example, A being part of the MINUS-formula (1) entails that there are two configurations σ_i and σ_j such that e is given and $C{*}d$ is not given in both σ_i and σ_j, while σ_i features A and B and σ_j features a and b. Only if such a difference-making pair $\langle \sigma_i, \sigma_j \rangle$ exists is A indispensable to account for B. Analogously, (1) being a MINUS-formula entails that there exist difference-making pairs for all other (appearances of) factor values in (1).

To define causation in terms of Boolean difference-making, an additional condition is needed because not all MINUS-formulas faithfully represent causation. Complete redundancy elimination is relative to the set of analyzed factors $\mathbf{F}$, meaning that factor values contained in a MINUS-formula relative to some $\mathbf{F}$ may fail to be part of a MINUS-formula relative to supersets of $\mathbf{F}$ (Baumgartner 2013). In other words, by adding further factors to the analysis, factor values that originally appeared to be non-redundant to account for an outcome can turn out to be redundant after all. Hence, a *permanence* condition needs to be imposed: only factor values that are permanently non-redundant, meaning that cannot be rendered redundant by expanding factor sets, are causally relevant.

These considerations yield the following definition of causation:

Causal Relevance (MINUS) X is causally relevant to Y if, and only if, (I) X is part of a MINUS-formula of Y relative to some factor set $\mathbf{F}$ and (II) X remains part of a MINUS-formula of Y across all expansions of $\mathbf{F}$.

Two features of the (MINUS) definition make it particularly well suited for the analysis of structures affected by conjunctivity and disjunctivity. First, (MINUS) does not require that

causes and effects are pairwise dependent. The following is a well-formed MINUS-formula expressing the flow model from the introduction: $S{*}C \ + \ s{*}c \ \leftrightarrow \ F$. As shown in Table 1, ideal data generated from that model feature no pairwise dependencies. Nonetheless, if, say, high skills are permanently non-redundant to account for flow in combination with high challenges, they are causally relevant for flow subject to (MINUS), despite being uncorrelated with flow. Second, MINUS-formulas whose elements satisfy the permanence constraint not only identify causally relevant factor values but also place a Boolean ordering over these causes, such that conjunctivity and disjunctivity can be directly read off their syntax. Take the following complex MINUS-formula:

$$(A{*}b \ + \ a{*}B \ \leftrightarrow \ C) * (C{*}f \ + \ D \ \leftrightarrow \ E) \tag{2}$$

Causally interpreting (2) against the background of (MINUS) entails these causal ascriptions:

1. the factor values listed on the left-hand sides of "↔" are causally relevant for the factor values on the right-hand sides;
2. A and b are jointly relevant to C and located on a causal path that differs from the path on which the jointly relevant a and B are located; C and f are jointly relevant to E and located on a path that differs from D's path;
3. there is a causal chain from $A{*}b$ and $a{*}B$ via C to E.

Zhang and Zhang (2023) have recently proposed a fourth non-redundancy constraint on MINUS-formulas. It is possible that configurational data entail multiple atomic MINUS-formulas with the same factor value on the right-hand side. Let $\Psi_1 \leftrightarrow Y$ and $\Psi_2 \leftrightarrow Y$ be two such formulas, and let $\mathbf{F}_{\Psi_1}$ and $\mathbf{F}_{\Psi_2}$ be the sets of factors appearing in Ψ_1 and Ψ_2. If one of these sets is a proper subset of the other, one of the two formulas accounts for Y on the basis of at least one fewer factor than the other. The surplus factor is redundant, as Y can be expressed as a Boolean function without it. Accordingly, Zhang and Zhang (2023) require that an atomic MINUS-formula $\Psi \leftrightarrow Y$ be *functionally minimal*: $\mathbf{F}_{\Psi}$ must not contain a proper subset $\mathbf{F}'_{\Psi}$ such that Y can be expressed as a Boolean function of $\mathbf{F}'_{\Psi}$. Functional minimality differs from both minimal sufficiency and minimal necessity: minimal sufficiency eliminates redundant conjuncts from a sufficient condition and minimal necessity eliminates redundant disjuncts from a necessary condition, whereas functional minimality eliminates redundant factors from the factor set on the basis of which the outcome is modeled. The **cna** package makes the functional minimality criterion available as an option (see section 5.5 below).

2.4. Inferring MINUS causation from data

Inferring MINUS causation from data faces various challenges. First, as anticipated in section 1, causal structures for which conjunctivity and disjunctivity hold cannot be uncovered by scanning data for dependencies between pairs of factor values and suitably combining dependent pairs. Instead, discovering MINUS causation requires searching for dependencies between complex Boolean functions of exogenous factors and outcomes. But the space of Boolean functions over more than five factors is so vast that it cannot be exhaustively scanned. Hence, algorithmic strategies are needed to purposefully narrow down the search.

Second, condition (MINUS.II) is not comprehensively testable. Once a MINUS-formula of an outcome Y comprising a factor value X has been inferred from data δ, the question arises

whether the non-redundancy of X in accounting for Y is an artifact of δ, due, for example, to the uncontrolled variation of confounders, or whether it is genuine and persists when further factors are taken into consideration. But in practice, expanding the set of factors is only feasible within narrow confines. To make up for the impossibility of testing (MINUS.II), the analyzed data δ should be collected in such a way that Boolean dependencies in δ are not induced by an uncontrolled variation of latent causes but by the measured factors themselves. If the dependencies in δ are not artefacts of latent causes, they cannot be neutralized by factor set expansions, meaning they are permanent and, hence, causal. It follows that in order for it to be guaranteed that causal inferences drawn from δ are error-free, δ must meet very high quality standards. In particular, the uncontrolled causal background of δ must be *homogeneous* (Baumgartner and Thiem 2020, 286):

Homogeneity The unmeasured causal background of data δ is homogeneous if, and only if, latent causes not connected to the outcome(s) in δ on causal paths via the measured exogenous factors (so-called *off-path* causes) take constant values (i.e. do not vary) in the cases recorded in δ.

However, third, real-life data often do not meet very high quality standards. Rather, they tend to be *fragmented* and *noisy*. Data are fragmented when not all possible configurations of the analyzed factors are observed, and they are noisy when they contain cases incompatible with the data-generating structure. The degree of fragmentation corresponds to the ratio of configurations of exogenous factors that are compatible with the data-generating structure but missing from the data, due to practical limitations of data collection. Noise, in turn, is measurable as the ratio of cases in the data that are incompatible with the data-generating structure. Noise can be induced, for instance, by measurement error or limited control over latent causes, i.e. confounding. In the presence of fragmentation and noise, there typically are no strict Boolean sufficiency or necessity relations in the data. In consequence, methods of MINUS discovery have to carefully evaluate the available evidence on sufficiency and necessity relations; and they can only approximate strict MINUS structures by fitting their models more or less closely to the data using suitable parameters and thresholds of model fit (De Souter 2024; De Souter and Baumgartner 2025; for details see section 3.2 below). Moreover, noise stemming from the uncontrolled variation of latent causes gives rise to homogeneity violations, with the result that inferences to MINUS causation are not guaranteed to be error-free. In order to nonetheless distill causal information from noisy data, strategies for avoiding over- and underfitting and estimating the error risk are needed (see Parkkinen and Baumgartner 2023).

Fourth, according to the MINUS theory, the inference to *causal irrelevance* is much more demanding than the inference to causal relevance. Establishing that X is a MINUS cause of Y requires demonstrating the existence of at least one context with a constant background in which a difference in X is associated with a difference in Y, whereas establishing that X is *not* a MINUS cause of Y requires demonstrating the *non-existence* of such a context, which is impossible on the basis of the non-exhaustive data samples that are typically analyzed in real-life studies. Correspondingly, the fact that, say, G does not appear in (2) does not imply that G is causally irrelevant to C or E. The non-inclusion of G simply means that the data from which (2) has been derived do not contain evidence for the causal relevance of G. However, future research having access to additional data might reveal the existence of a difference-making context for G and, hence, entail the causal relevance of G to C or E after

all.

Finally, on a related note, as a result of the common fragmentation of real-life data δ MINUS-formulas inferred from δ cannot be expected to completely reflect the causal structure generating δ. That is, MINUS-formulas inferred from δ are inevitably going to be *incomplete*. They only detail those causally relevant factor values along with those conjunctive, disjunctive, and sequential groupings for which δ contains difference-making evidence. What difference-making evidence is contained in δ not only depends on the cases recorded in δ but, when δ is noisy, also on the tuning thresholds imposed to approximate strict Boolean dependency structures; relative to some such tuning settings an association between X and Y may pass as a sufficiency or necessity relation whereas relative to another setting it will not. Hence, the inference to MINUS causation is sensitive to the chosen tuning settings, to the effect that choosing different settings is often going to be associated with changes in inferred MINUS-formulas.

Some variance (but not all) in inferred MINUS-formulas is unproblematic. Two different MINUS-formulas $\mathbf{m}_i$ and $\mathbf{m}_j$ derived from δ using different tuning settings are in no disagreement if $\mathbf{m}_i$ and $\mathbf{m}_j$ are related in terms of the submodel relation:

Submodel relation A MINUS-formula $\mathbf{m}_i$ is a *submodel* of another MINUS-formula $\mathbf{m}_j$ if, and only if, the causal ascriptions entailed by $\mathbf{m}_i$ are a subset of the causal ascriptions entailed by $\mathbf{m}_j$.

If $\mathbf{m}_i$ is a submodel of $\mathbf{m}_j$, $\mathbf{m}_j$ is a *supermodel* of $\mathbf{m}_i$. All of $\mathbf{m}_i$'s causal ascriptions are contained in its supermodels' ascriptions, and $\mathbf{m}_i$ contains the causal ascriptions of its own submodels. The submodel relation is reflexive: every model is a submodel (and supermodel) of itself; moreover, if $\mathbf{m}_i$ and $\mathbf{m}_j$ are submodels of one another, then $\mathbf{m}_i$ and $\mathbf{m}_j$ are identical. Most importantly, if two MINUS-formulas related by the submodel relation are not identical, they can be interpreted as describing the same causal structure at different levels of detail.

Before we turn to the **cna** package, a terminological note is required. In the literature on configurational comparative methods it has become customary to refer to the models produced by the methods as *solution formulas*. To mirror that convention, the **cna** package refers to atomic MINUS-formulas inferred from data by CNA as *atomic solution formulas*, ***asf***, for short, and to complex MINUS-formulas inferred from data as *complex solution formulas*, ***csf***. For brevity, we will henceforth mainly use the shorthands *asf* and *csf*.

3. The input of CNA

The goal of CNA is to output all *asf* and *csf* within provided bounds of model complexity that fit an input data set relative to provided tuning settings, in particular, fit thresholds. The algorithm performing this task in the **cna** package is implemented in the function `cna()`. Its most important arguments are:

```
cna(x, outcome = TRUE, con = 1, cov = 1, maxstep = c(3, 4, 10),
    measures = c("standard consistency", "standard coverage"),
    ordering = NULL, strict = FALSE, exclude = character(0), notcols = NULL,
    what = if (suff.only) "m" else "ac", details = FALSE, suff.only = FALSE,
```

```
    acyclic.only = FALSE, cycle.type=c("factor","value"),
    func.minimal.only = FALSE)
```

This section explains most of these inputs and introduces some auxiliary functions. The arguments `what`, `acyclic.only`, `cycle.type`, and `func.minimal.only` will be discussed in section 5.

3.1. Data

Data δ processed by CNA have the form of $m \times k$ matrices, where m is the number of units of observation (cases) and k is the number of measured factors. δ can either be of type "crisp-set" (cs), "multi-value" (mv) or "fuzzy-set" (fs). Data that feature cs factors only are cs. If the data contain at least one mv factor, they count as mv. Data with at least one factor with non-integer values from the unit interval $[0, 1]$ are treated as fs.[6] Examples of each data type are given in Table 3. Raw data collected in a study typically need to be suitably calibrated before they can be fed to `cna()`. We do not address the calibration problem here because it is the same for CNA as for QCA, in which context it has been extensively discussed, for example, by Thiem and Duşa (2013) or Schneider and Wagemann (2012). The R packages **QCA**, and **SetMethods** provide all tools necessary for data calibration.

Calibrated data are given to the `cna()` function via the argument `x`, which must be a data frame or an object of class "configTable" as output by the `configTable()` function (see section 3.1.1 below). The **cna** package contains a number of example data sets from published studies, `d.autonomy`, `d.educate`, `d.irrigate`, `d.jobsecurity`, `d.minaret`, `d.pacts`, `d.pban`, `d.performance`, `d.volatile`, `d.women`, and one simulated data set, `d.highdim`. For details on their contents and sources, see the **cna** reference manual. After having loaded the **cna** package, all of these data sets are available for processing:

```
R> library(cna)
R> cna(d.educate)
R> cna(d.women)
```

	A	B	C	D
c_1	0	0	0	0
c_2	0	1	0	0
c_3	1	1	0	0
c_4	0	0	1	0
c_5	1	0	0	1
c_6	1	0	1	1
c_7	0	1	1	1
c_8	1	1	1	1

(a) cs data

	A	B	C	D
c_1	1	3	3	1
c_2	2	2	1	2
c_3	2	1	2	2
c_4	2	2	2	2
c_5	3	3	3	2
c_6	2	4	3	2
c_7	1	3	3	3
c_8	1	4	3	3

(b) mv data

	A	B	C	D	E
c_1	0.37	0.30	0.16	0.06	0.25
c_2	0.89	0.39	0.64	0.09	0.03
c_3	0.06	0.61	0.92	0.37	0.15
c_4	0.65	0.93	0.92	0.18	0.93
c_5	0.08	0.08	0.12	0.86	0.91
c_6	0.70	0.02	0.85	0.91	0.97
c_7	0.04	0.72	0.76	0.90	0.68
c_8	0.81	0.96	0.89	0.72	0.82

(c) fs data

Table 3: Data types processable by CNA.

[6] Note, first, that factors calibrated at crisp-set thresholds may appear with unsuitably extreme values if the data as a whole are treated as fs due to some fs factor, and second, that mixing mv and fs factors in one analysis is not possible.

Configuration tables

To facilitate the reviewing of data, the function `configTable(x, case.cutoff = 0)` assembles cases with identical configurations in one row of a so-called *configuration table*. A configuration table is not to be confused with what is called a *truth table* in QCA. While a QCA truth table indicates for every configuration of all exogenous factors (i.e. for every minterm) whether it is sufficient for the outcome, a CNA configuration table does not express relations of sufficiency but simply provides a compact representation of the data that lists all configurations exactly once and adds a column indicating how many instances (cases) of each configuration are contained in the data.

The first argument `x` of `configTable()` is a data frame or matrix. The function then merges multiple rows of `x` featuring the same configuration into one row, such that each row of the resulting table corresponds to one determinate configuration of the factors in `x`. The number of occurrences of a configuration and an enumeration of the cases instantiating it are saved as attributes "n" and "cases", respectively.

```
R> configTable(d.women)

configTable of type "cs"
            ES QU WS WM LP WNP | n.obs
SE           1  1  1  0  0   1 |     1
FI           1  0  1  0  0   1 |     1
IS,NO        1  1  1  1  1   1 |     2
DK           1  0  1  1  1   1 |     1
BE,NL        1  1  0  1  1   1 |     2
ES           1  1  0  1  0   1 |     1
AT           1  1  0  0  1   1 |     1
NZ           0  0  0  1  1   1 |     1
DE           0  1  0  1  1   1 |     1
CH,GR,PT     1  1  0  0  0   0 |     3
AU,FR,GB,IE  0  1  0  1  0   0 |     4
LU           1  0  0  0  1   0 |     1
CA,US        0  0  0  1  0   0 |     2
IT           0  1  0  0  0   0 |     1
Total no.of.cases: 22
```

The second argument `case.cutoff` allows for setting a minimum frequency cutoff determining that configurations with fewer instances in the data are not included in the configuration table and the ensuing analysis. For instance, `configTable(x, case.cutoff = 3)` entails that configurations that are instantiated in less than 3 cases are excluded.

Configuration tables produced by `configTable()` can be directly passed to `cna()`. Moreover, as configuration tables generated by `configTable()` are objects that are very particular to the **cna** package, the function `ct2df()` is available to transform configuration tables back into ordinary R data frames.

```
R> pact.ct <- configTable(d.pacts, case.cutoff = 2)
R> ct2df(pact.ct)
```

Data simulations

The **cna** package provides extensive functionalities for data simulations—which, in turn, are essential for inverse search trials that benchmark CNA's output (see section 7). In a nutshell, the functions `allCombs()` and `full.ct()` generate the space of all logically possible configurations over a given set of factors, `selectCases()` selects, from this space, the configurations that are compatible with a data-generating causal structure, which, in turn, can be randomly drawn by `randomAsf()` and `randomCsf()`, `makeFuzzy()` fuzzifies that data, and `some()` randomly selects cases, for instance, to produce data fragmentation.

More specifically, `allCombs(x)` takes an integer vector `x` as input and generates a configuration table featuring all possible value configurations of `length(x)` factors—the first factor having `x[1]` values, the second `x[2]` values etc. The factors are labeled using capital letters in alphabetical order. Analogously, but more flexibly, `full.ct(x)` generates a configuration table with all logically possible value configurations of the factors defined in the input `x`, which can be a configuration table, a data frame, an integer, a list specifying the factors' value ranges, or a character vector featuring all admissible factor values.

```
R> allCombs(c(2, 2, 2)) - 1
R> allCombs(c(3, 4, 5))
R> full.ct("A + B*c")
R> full.ct(6)
R> full.ct(list(A = 1:2, B = 0:1, C = 1:4))
```

The input of `selectCases(cond, x)` is a character string `cond` specifying a Boolean function, which typically (but not necessarily) expresses a data-generating MINUS structure, as well as, optionally, a data frame or configuration table `x`. If `x` is specified, the function selects the cases that are compatible with `cond` from `x`; if `x` is not specified, it selects from `full.ct(cond)`. It is possible to randomly draw `cond` using `randomAsf(x)` or `randomCsf(x)`, which generate random atomic and complex solution (i.e. MINUS-)formulas, respectively, from a data frame or configuration table `x`.

```
R> dat1 <- allCombs(c(2, 2, 2)) - 1
R> selectCases("A + B <-> C", dat1)
R> selectCases("(h*F + B*C*k + T*r <-> G)*(A*b + H*I*K <-> E)")
R> target <- randomCsf(full.ct(6))
R> selectCases(target)
```

`makeFuzzy(x, fuzzvalues = c(0, 0.05, 0.1))` simulates fuzzy-set data by transforming a data frame or configuration table `x` consisting of *cs* factors into an *fs* configuration table. To this end, the function adds values selected at random from the argument `fuzzvalues` to the 0's and subtracts them from the 1's in `x`. `fuzzvalues` is a numeric vector of values from the interval [0,1].

```
R> makeFuzzy(selectCases("Hunger + Heat <-> Run"),
+     fuzzvalues = seq(0, 0.4, 0.05))
```

Finally, `some(x, n = 10, replace = TRUE)` randomly selects `n` cases from a data frame or configuration table `x`, with or without replacement. If `x` features all configurations that are

compatible with a data-generating structure and `n < nrow(x)`, the data frame or configuration table issued by `some()` is fragmented, meaning it does not contain all empirically possible configurations. If `n > nrow(x)`, data of large sample sizes can be generated featuring multiple instances of the empirically possible configurations.

```
R> dat3 <- allCombs(c(3, 4, 5))
R> dat4 <- selectCases("A=1*B=3 + A=3 <-> C=2", configTable(dat3))
R> some(dat4, n = 10, replace = FALSE)
R> some(dat4, n = 1000, replace = TRUE)
```

3.2. Evaluating sufficiency and necessity

As we have seen in section 2.4, real-life data tend to be affected by fragmentation and noise, in the presence of which strictly sufficient or necessary conditions for an outcome often do not exist. Accordingly, methods for MINUS discovery have to carefully evaluate what inferences on sufficiency and necessity relations the available evidence warrants. In particular, it must be assessed whether the nonexistence of strictly sufficient or necessary conditions is more likely due to deficiencies in data collection and lack of control over background influences, or whether it is indicative of the absence of causal relations. The former holds if, and only if, strict sufficiency and necessity would have been observed had the data been ideal, that is, non-fragmented and noise-free. Various measures are available to help evaluate whether that hypothetical holds or not. As the implication operator underlying the notions of sufficiency and necessity is defined differently in classical and fuzzy logic, the evaluation measures for *cs* and *mv* data have different formal definitions than the measures for fs data. But since they are very closely related conceptually, the measures have identical names in the *cs*/*mv* and fs contexts. Their names reflect the fact that they have all evolved from *consistency* and *coverage*, which Ragin (2006) imported into the QCA protocol and which have proven serviceable to the purposes of CNA as well. This section first presents the variants of consistency and coverage suitable for *cs* and *mv* data, then turns to the fs case, and finally discusses the choice of evaluation measures and the setting of corresponding thresholds.

Crisp-set and multi-value data

To introduce the relevant evaluation measures for *cs* and *mv* data, we use Φ as a placeholder for a Boolean expression in DNF (see fn. 5), for example, A, $A{*}C$, or A=2∗C=3 + A=1∗C=1, while ϕ represents the negation of that DNF. Analogously, Y and y shall be placeholders for single factor values and their negations, for example, A and a. We call Φ the *antecedent* whose sufficiency or necessity for the *outcome* Y is to be assessed. Moreover, we use cardinality bars $|\ldots|$ to refer to the number of cases in the analyzed data δ satisfying the enclosed expression. For example, $|\Phi{*}Y|$ designates the number of cases in δ instantiating $\Phi{*}Y$ (i.e. both the antecedent and the outcome).

In versions of **cna** < 4.0, the only implemented sufficiency measure was *consistency*, which we give in two equivalent forms here—the first being the most commonly used form and the

second being the form that renders the measure's penalty term ($|\Phi{*}y|$) explicit.[7]

$$consistency(\Phi, Y) = \frac{|\Phi{*}Y|}{|\Phi|} = \frac{|\Phi{*}Y|}{|\Phi{*}Y| + |\Phi{*}y|}$$

Consistency is formally equivalent to what is known as *positive predictive value* or *precision* in various fields of machine learning. It considers all cases in δ featuring Φ and measures the proportion of them that satisfy $\Phi \rightarrow Y$, which are those that instantiate Y in addition to Φ. It penalizes the cases with Φ that violate $\Phi \rightarrow Y$, that is, cases with $\Phi{*}y$.

Even though this penalization schema makes good intuitive sense, consistency has two distinctive weaknesses (see De Souter 2024). First, as cases exhibiting Y cannot violate $\Phi \rightarrow Y$, consistency tends to be high in data δ with a high proportion of cases with Y, meaning with high outcome prevalence. Even if Φ and Y are entirely independent in δ, the consistency of Φ for Y is equal to the prevalence of Y (De Souter and Baumgartner 2025). In other words, when the prevalence of Y is high, consistency is high for every (arbitrary) Φ. But of course, the mere fact that most cases in δ instantiate Y is not evidence in favor of every Φ being sufficient for Y in the (hypothetical) ideal version of δ. Second, if cases with Y are rare, there are only few cases that could possibly corroborate that $\Phi \rightarrow Y$ holds (i.e. the few cases with Y), and if some of them are affected by noise, consistency plummets. In consequence, the chances that consistency can detect sufficiency satisfaction are low. Overall, consistency is too lenient when prevalence is high and it is overly strict when prevalence is low.

To address these weaknesses, **cna** 4.0 makes three new sufficiency measures available: *prevalence-adjusted consistency* (PA-consistency), *contrapositive consistency* (C-consistency), and *antecedent-adjusted C-consistency* (AAC-consistency).

$$PA\text{-}consistency(\Phi, Y) = \frac{|\Phi{*}Y|}{|\Phi{*}Y| + \frac{|Y|}{|y|} \cdot |\Phi{*}y|}$$

$$C\text{-}consistency(\Phi, Y) = \frac{|\phi{*}y|}{|\phi{*}y| + |\Phi{*}y|}$$

$$AAC\text{-}consistency(\Phi, Y) = \frac{|\phi{*}y|}{|\phi{*}y| + \frac{|\phi|}{|\Phi|} \cdot |\Phi{*}y|}$$

PA-consistency is a variant of consistency that is equivalent to calibrated precision as proposed by Siblini *et al.* (2020). It differs from consistency by the weight $\frac{|Y|}{|y|}$ attached to the penalty term $|\Phi{*}y|$ in the denominator. When $|Y| = |y|$, this weight is 1, to the effect that PA-consistency and consistency are equal. However, the weight increases as $|Y|$ increases relative to $|y|$, and it decreases as $|Y|$ decreases relative to $|y|$. As a result, $|\Phi{*}y|$ is penalized more strongly when prevalence is high and less strongly when prevalence is low.

C-consistency is equivalent to the measure of *specificity* used in machine learning. Its use as a sufficiency measure in CNA is based on the rule of contraposition, which states that $\Phi \rightarrow Y$ is logically equivalent to $y \rightarrow \phi$. In order for Φ to be sufficient for Y, the cases with y must exhibit ϕ. Accordingly, C-consistency penalizes the cases with y exhibiting Φ, which are the same cases penalized by consistency and exactly the cases violating $\Phi \rightarrow Y$ and

[7] The evaluation measures discussed in this section contain arithmetic sums. We symbolize sums with script-style "$+$", as opposed to the "$+$" used for Boolean disjunction.

$y \rightarrow \phi$. De Souter (2024) has shown that a combined use of consistency and C-consistency can increase model quality substantively. Still, even though C-consistency does not suffer from the weaknesses of consistency, it has its own limitations. It is too lenient when Φ is infrequent in δ, and it is overly strict when Φ is frequent (see De Souter and Baumgartner 2025 for details).

AAC-consistency addresses the limitations of C-consistency by adjusting the penalty term $|\Phi{*}y|$ in the denominator of C-consistency by the weight $\frac{|\phi|}{|\Phi|}$. If $|\phi| = |\Phi|$, this weight is 1, rendering AAC-consistency and C-consistency equal. However, the weight increases as $|\phi|$ increases relative to $|\Phi|$ and it decreases as $|\phi|$ decreases relative to $|\Phi|$, so that $|\Phi{*}y|$ is penalized more strongly when Φ is infrequent and less strongly when Φ is frequent.

The only necessity measure implemented in versions of **cna** < 4.0 was *coverage*, which, for reasons of transparency, we also write in two equivalent forms here.

$$coverage(\Phi, Y) = \frac{|\Phi{*}Y|}{|Y|} = \frac{|\Phi{*}Y|}{|\Phi{*}Y| + |\phi{*}Y|}$$

Coverage is equivalent to what is known as *sensitivity* or *recall* in machine learning. It considers all cases in δ featuring Y and measures the proportion of them that comply with the necessity of Φ for Y (i.e. $Y \rightarrow \Phi$), which are those that instantiate both Φ and Y, and it penalizes the cases exhibiting Y without Φ, that is, the cases with $\phi{*}Y$.

Despite following a sensible penalization scheme for necessity relations, coverage has weaknesses analogous to those of consistency (De Souter 2024). On the one hand, it tends to be high in data δ with a high proportion of cases featuring the antecedent Φ. Any (arbitrary) Φ that is independent of Y scores high on coverage, as long as it is given in a high proportion of cases in δ (De Souter and Baumgartner 2025). Plainly, though, that most cases instantiate Φ is not evidence in favor of Φ being necessary for Y in the (hypothetical) ideal version of δ. On the other hand, coverage is greatly susceptible to noise when the proportion of cases with the antecedent is low. As it penalizes cases with $\phi{*}Y$ in proportion to cases with $\Phi{*}Y$, only a few noisy cases with $\phi{*}Y$ pull down coverage significantly, irrespective of whether Φ is actually a cause of Y. So, coverage is too lenient when the proportion of Φ is high and too strict when that proportion is low.

Version 4.0 of **cna** provides three new necessity measures: *antecedent-adjusted coverage* (AA-coverage), *contrapositive coverage* (C-coverage), and *prevalence-adjusted C-coverage* (PAC-coverage).

$$AA\text{-}coverage(\Phi, Y) = \frac{|\Phi{*}Y|}{|\Phi{*}Y| + \frac{|\Phi|}{|\phi|} \cdot |\phi{*}Y|}$$

$$C\text{-}coverage(\Phi, Y) = \frac{|\phi{*}y|}{|\phi{*}y| + |\phi{*}Y|}$$

$$PAC\text{-}coverage(\Phi, Y) = \frac{|\phi{*}y|}{|\phi{*}y| + \frac{|y|}{|Y|} \cdot |\phi{*}Y|}$$

As regular coverage is too lenient when the proportion of Φ is high and too strict when that proportion is low, AA-coverage adds the weight $\frac{|\Phi|}{|\phi|}$ to the penalty $|\phi{*}Y|$. This makes AA-coverage stricter than coverage when the antecedent proportion is high and more lenient than coverage when it is low, thereby mitigating the limitations of coverage.

C-coverage is equivalent to *negative predictive value* (NPV). Its use as a necessity measure is justified by the rule of contraposition, which guarantees that Φ is necessary for Y if, and only if, y is necessary for ϕ. Both necessity relations are violated by cases with $\phi{*}Y$ (and by no others). Correspondingly, C-coverage penalizes the cases exhibiting $\phi{*}Y$ by measuring the proportion of cases with ϕ that instantiate y. It does not suffer from the limitations of coverage, but it has its own weaknesses. It is too lenient when prevalence is low, and it is too strict when prevalence is high (De Souter and Baumgartner 2025).

To alleviate these problems of C-coverage, PAC-coverage adjusts for prevalence by adding the weight $\frac{|y|}{|Y|}$ to the penalty term $|\phi{*}Y|$. This weight ensures that, when prevalence is low, cases with $\phi{*}Y$ are penalized more strongly, and less strongly when prevalence is high.

Fuzzy-set data

Contrary to the cs/mv measures, the fs measures are not defined in terms of case cardinalities but based on sums over membership scores in the data δ—for example, $\sum_{i=1}^{n}\min(\Phi_i, Y_i)$, which is the sum over $\min(\Phi_i, Y_i)$[8] in all n cases of δ. For convenience, we will shorten sums by dropping running indices, meaning that $\sum_{i=1}^{n}\min(\Phi_i, Y_i)$ becomes $\sum\min(\Phi, Y)$. To maximize transparency with respect to penalization, we again provide two equivalent forms for the fs versions of the standard sufficiency and necessity measures:

$$\begin{aligned} \mathit{consistency}(\Phi, Y) &= \frac{\sum\min(\Phi, Y)}{\sum\Phi} \\ &= \frac{\sum\min(\Phi, Y)}{\sum(\min(\Phi, Y) + \min(\Phi, y) - \min(\Phi, \phi, Y, y))} \end{aligned}$$

$$\begin{aligned} \mathit{coverage}(\Phi, Y) &= \frac{\sum\min(\Phi, Y)}{\sum Y} \\ &= \frac{\sum\min(\Phi, Y)}{\sum(\min(\Phi, Y) + \min(\phi, Y) - \min(\Phi, \phi, Y, y))} \end{aligned}$$

Subtracting $\min(\Phi, \phi, Y, y)$ from the penalty terms in the denominators of consistency and coverage is needed to correct for the double-counting due to cases with non-zero membership scores in all of Φ, ϕ, Y, and y (for more details see De Souter 2024, 24). Such corrections are not necessary in cs and mv measures, as values of cs and mv factors are mutually exclusive, meaning that $\min(\Phi, \phi, Y, y)$ is always 0.

The limitations of standard consistency and coverage for sufficiency and necessity evaluation in fs data are analogous to those in cs and mv data. To address them, version 4.0 of **cna** provides fs variants of PA-consistency, C-consistency, AAC-consistency, AA-coverage, C-coverage, and PAC-coverage. The fs measures are defined as follows:

$$\mathit{PA\text{-}consistency}(\Phi, Y) = \frac{\sum\min(\Phi, Y)}{\sum\min(\Phi, Y) + \frac{\sum Y}{\sum y}\cdot\sum(\min(\Phi, y) - \min(\Phi, \phi, Y, y))}$$

$$\mathit{C\text{-}consistency}(\Phi, Y) = \frac{\sum\min(\phi, y)}{\sum(\min(\phi, y) + \min(\Phi, y) - \min(\Phi, \phi, Y, y))}$$

[8] The minimum function is the fs rendering of conjunction, see p. 6 above.

$$AAC\text{-}consistency(\Phi, Y) = \frac{\sum \min(\phi, y)}{\sum \min(\phi, y) + \frac{\sum \phi}{\sum \Phi} \cdot \sum(\min(\Phi, y) - \min(\Phi, \phi, Y, y))}$$

$$AA\text{-}coverage(\Phi, Y) = \frac{\sum \min(\Phi, Y)}{\sum \min(\Phi, Y) + \frac{\sum \Phi}{\sum \phi} \cdot \sum(\min(\phi, Y) - \min(\Phi, \phi, Y, y))}$$

$$C\text{-}coverage(\Phi, Y) = \frac{\sum \min(\phi, y)}{\sum(\min(\phi, y) + \min(\phi, Y) - \min(\Phi, \phi, Y, y))}$$

$$PAC\text{-}coverage(\Phi, Y) = \frac{\sum \min(\phi, y)}{\sum \min(\phi, y) + \frac{\sum y}{\sum Y} \cdot \sum(\min(\phi, Y) - \min(\Phi, \phi, Y, y))}$$

Choosing measures and setting thresholds

Sufficiency and necessity measures play a twofold role in CNA: on the one hand, they are key in CNA's model-building algorithm (see section 4), and on the other, they are used to select among multiple model candidates output by that algorithm (section 6.3). When the analyzed data are noisy and/or fragmented, strictly sufficient and necessary conditions may not exist for scrutinized outcomes. But associations that score reasonably high on chosen sufficiency and necessity measures may be acceptable as sufficiency or necessity relations nonetheless, on the grounds that they would have been relations of strict sufficiency or necessity had the data been ideal. By lowering the thresholds for associations to pass the sufficiency or necessity test and accepting the concomitant error risk, it becomes possible to extract causal information from data despite the presence of noise and fragmentation.

The `measures` argument of the `cna()` function selects the sufficiency and necessity measures to be employed for model-building. It expects a vector of length two as input, where the first value specifies the variant of consistency to be used as sufficiency measure and the second the variant of coverage to be used as necessity measure. The measures can be identified by their full names or by their aliases from the following list.

```
R> showConCovMeasures()

standard consistency
    aliases: "scon", "s-con"
standard coverage
    aliases: "scov", "s-cov"
contrapositive consistency
    aliases: "ccon", "c-con"
contrapositive coverage
    aliases: "ccov", "c-cov"
prevalence-adjusted consistency
    aliases: "PAcon", "PA-con"
antecedent-adjusted coverage
    aliases: "AAcov", "AA-cov"
antecedent-adjusted contrapositive consistency
```

```
    aliases: "AACcon", "AAC-con"
prevalence-adjusted contrapositive coverage
    aliases: "PACcov", "PAC-cov"
```

The default is `measures = c("standard consistency", "standard coverage")`, which can be abbreviated as `measures = c("scon", "scov")`. For ideal data, all combinations of sufficiency and necessity measures yield the same output (at maximal thresholds), meaning that any combination is as good as any other. However, for noisy and fragmented data, different combinations may produce different models. These differences tend to increase with the degree to which the value distributions of the factors in the data are imbalanced. Which measure combinations to use for which data scenarios is a matter of ongoing research. The results of De Souter and Baumgartner (2025) show that the measure combination producing the most reliable models for *cs* data is *PA-consistency* for sufficiency evaluation and *PAC-coverage* for necessity evaluation, that is, `measures = c("PAcon", "PACcov")`. Their results also show that when the prevalence of the outcome is balanced within the mid-range (i.e. above 0.3 and below 0.7), all measure combinations perform similarly well. Unfortunately, no analogous studies have been conducted for *mv* and *fs* data yet. Hence, for these data types, the default measure combination `measures = c("scon", "scov")` is recommended at present. Still, for all data types and scenarios, different measure combinations can be used for cross-validation purposes. De Souter (2024) has shown that models scoring similarly high on consistency and C-consistency as well as on coverage and C-coverage are more likely to be correct than models scoring high on only consistency and coverage.

The `details` argument can be used to obtain the scores of the returned *asf* and *csf* on any of the available evaluation measures, irrespective of whether they are used for model-building or not. The argument expects a character vector specifying the measures of interest, which can be identified by their full names or by their aliases.

```
R> cna(d.women, measures = c("PAcon", "PACcov"), details = c("scon", "scov",
+     "ccon", "ccov", "AAcov", "AACcon"))
```

The function behind the `details` argument is also available as stand-alone function `detailMeasures()`, which is explained in the **cna** reference manual.

Thresholds (from the interval $[0, 1]$) for the chosen variants of consistency and coverage can be given to the `cna()` function using the arguments `con` and `cov`. The `con` argument sets the consistency (or sufficiency) threshold for minimally sufficient conditions (*msc*), for *asf* and *csf*, while `cov` sets the coverage (or necessity) threshold for *asf* and *csf* (no coverage threshold is imposed on *msc*). The default numeric value for all thresholds is 1, which corresponds to strict Boolean sufficiency and necessity. Contrary to QCA, which often returns solutions that do not comply with the chosen consistency threshold and which does not impose a coverage threshold at all, CNA uses the selected sufficiency and necessity measures as authoritative model-building criteria such that, if they are not met, CNA abstains from issuing solutions. That means, if the default thresholds are used, `cna()` will only output strictly sufficient *msc*, *asf*, and *csf* and only strictly necessary *asf* and *csf*.

If the data are noisy, the default thresholds will typically not yield any solution formulas. In such cases, `con` and `cov` may be suitably lowered. By lowering `con` below 1 in a *cs* analysis, `cna()` is given permission to treat Φ as sufficient for Y, even though there are some cases

with Φ*y in the data. Or by lowering `cov` in an fs analysis, `cna()` is allowed to treat Φ as necessary for Y, even though some cases have higher membership scores on Y than on Φ, meaning that the sum of the membership scores in Y over all cases in the data exceeds the sum of the membership scores in $\min(\Phi, Y)$.

Determining the optimal values to which `con` and `cov` should be lowered in a specific discovery context is a delicate task. On the one hand, CNA faces a severe overfitting risk when the data contain configurations incompatible with the data-generating structure, meaning that `con` and `cov` must not be set too high (i.e. too close to 1). On the other hand, the lower `con` and `cov` are set, the less complex and informative CNA's output will be, that is, the more CNA's purpose of uncovering causal complexity will be undermined. To find a suitable balance between over- and underfitting, Parkkinen and Baumgartner (2023) systematically re-analyze the data at all `con` and `cov` settings in the interval $[0.7, 1]$, collect all solutions resulting from such a re-analysis series in a set $\mathbf{M}$, and select the solution formulas with the most sub- and supermodels in $\mathbf{M}$. These are the solutions with the highest *overlap in causal ascriptions* with the other solutions in $\mathbf{M}$. They are the most *robust* solutions inferable from the data. This approach to robustness scoring is implemented in the function `frscored_cna(x, fit.range, granularity)` of the R package **frscore** (Parkkinen and Baumgartner 2025).[9] The function `frscored_cna()` accepts all arguments of `cna()`, except for `con` and `cov`. It analyzes the data `x` with the sufficiency and necessity measures selected by `measures` at all threshold combinations in the interval `fit.range` with increments specified by `granularity`, and it scores the resulting models based on their robustness.

```
R> library(frscore)
R> frscored_cna(d.pban, fit.range = c(1, 0.8), granularity = 0.1)
```

If the analyst does not want to conduct a whole robustness analysis, reasonable non-perfect threshold settings are `con` = `cov` = 0.8 or 0.75. To illustrate, `cna()` does not build solutions for the fs data named `d.jobsecurity` at the following `con` and `cov` thresholds (the argument `outcome` is explained in section 3.3 below):

```
R> cna(d.jobsecurity, outcome = "JSR", con = 1, cov = 1)
R> cna(d.jobsecurity, outcome = "JSR", con = .9, cov = .9)
```

But if `con` and `cov` are set to 0.75, 20 solutions are returned with the default `measures` and 17 solutions with `measures = c("PAcon", "PACcov")`—the latter set of solutions being a subset of the former:

```
R> cna(d.jobsecurity, outcome = "JSR", con = .75, cov = .75)
R> cna(d.jobsecurity, outcome = "JSR", con = .75, cov = .75,
+     measures = c("PAcon", "PACcov"))
```

In the presence of noise, it is generally advisable to vary the `con` and `cov` settings to some degree to assess how sensitive the model space is to changes in tuning parameters and to evaluate the overlap in causal ascriptions across different solutions. Less complex solutions

[9] In addition, the R package **cnaOpt** (Ambühl and Baumgartner 2025) provides functions for finding the maximal consistency and coverage scores obtainable from a given data set and for identifying models reaching those scores. For a discussion of possible applications of maximal scores see (Baumgartner and Ambühl 2021).

are generally preferable to more complex ones, and solutions with more overlap are preferable to solutions with less overlap. If the sufficiency and necessity scores of resulting solutions can be increased by raising the `con` and `cov` settings without, at the same time, disproportionately increasing the solutions' complexity, solutions with higher fit are preferable to solutions with lower fit. But if an increase in fit comes with a substantive increase in model complexity, less complex models with lower fit are to be preferred (to avoid overfitting).

3.3. Outcome, ordering, and exclude

In principle, the `cna()` function does not need to be told which factors in the data `x` have values that are endogenous (i.e. are outcomes). It attempts to infer that from `x`. However, in ordinary research contexts, analysts do not start from scratch. They often possess some prior theoretical or causal knowledge about an investigated structure that allows them to, for example, identify potential outcomes or to exclude certain causal relationships. The `cna()` function provides three arguments—`outcome`, `ordering`, and `exclude`—through which prior causal information can be supplied to efficiently constrain the search space. This section introduces these arguments and their interplay.

Prior knowledge about which factors have values that can figure as outcomes can be given to `cna()` via the argument `outcome`, which takes as input a character vector specifying one or several factor values that are to be considered as potential outcome(s). For *cs* and *fs* data, factor values are expressed by upper and lower cases (e.g. `outcome = c("A", "b")`), for *mv* data, they are expressed by the "Factor=value" notation (e.g. `outcome = c("A=1","B=3")`). The default is `outcome = TRUE`, which means that values of all factors in `x` are potential outcomes. For example, the following function call determines that of all 9 factors in `d.volatile`, only $VO2$, i.e. VO2 taking the value 1, is a potential outcome, meaning that `cna()` does not attempt to model values of any other factor values as outcomes:

```
R> cna(d.volatile, outcome = "VO2")
```

When the data `x` contain multiple potential outcomes, it may moreover be known that these outcomes are causally ordered in a certain way, to the effect that some of them are causally upstream of the others. Such information can be given to CNA via a *causal ordering*, which is a relation A $\prec$ C (defined on the factors in `x`) entailing that values of C cannot cause values of A (e.g. because instances of A occur temporally before instances of C). That is, an ordering excludes certain causal dependencies but does not stipulate any. The corresponding argument, `ordering`, takes as value a character string. For example, `ordering = "A, B < C"` determines that factor C is causally located after A and B, meaning that values of C are not potential causes of values of A and B. The latter are located on the same level of the ordering, for A and B are unrelated by $\prec$, whereas C is located on a level that is downstream of the A, B-level. If an ordering is provided, `cna()` only searches for MINUS-formulas in accordance with the ordering. An ordering does not need to explicitly mention all factors in `x`. If only a subset of the factors are assigned to `ordering`, the non-included factors are entailed to be upstream of the included ones. Hence, `ordering = "C"` means that C is located downstream of all other factors in `x`.

To further specify the causal ordering, the logical argument `strict` is available. It determines whether the elements of one level in an ordering can be causally related or not. For example, if `ordering = "A, B < C"` and `strict = TRUE`, then values of A and B are excluded to be

causally related and `cna()` skips corresponding tests. By contrast, if `ordering = "A, B < C"` and `strict = FALSE`, then `cna()` also searches for dependencies among values of A and B.

Let us illustrate this with the data `d.autonomy`. Relative to the following function call, which stipulates that values of AU cannot cause values of EM, SP, and CO and that the latter factors are not mutually causally related, `cna()` infers that SP is causally relevant to AU:[10]

```
R> dat.aut.1 <- d.autonomy[15:30, c("AU","EM","SP","CO")]
R> ana.aut.1 <- cna(dat.aut.1, ordering = "EM, SP, CO < AU", strict = TRUE,
+     con = .9, cov = .9)
R> printCols <- c("condition", "con", "cov")
R> csf(ana.aut.1)[printCols]
```

```
  condition   con   cov
1 SP <-> AU 0.935 0.915
Measures:
  con: standard consistency
  cov: standard coverage
```

If we set `strict` to `FALSE` and, thereby, allow for causal dependencies among values of EM, SP, and CO, it turns out that SP not only causes AU, but, on another causal path, also makes a difference to EM:

```
R> ana.aut.2 <- cna(dat.aut.1, ordering = "EM, SP, CO < AU", strict = FALSE,
+     con = .9, cov = .9)
R> csf(ana.aut.2)[printCols]
```

```
  condition                       con   cov
1 (SP <-> AU)*(SP + CO <-> EM) 0.912 0.915
Measures:
  con: standard consistency
  cov: standard coverage
```

The arguments `ordering` and `outcome` interact closely. It is often not necessary to specify both of them. For example, `ordering = "C", strict = TRUE` is equivalent to `outcome = "C"`. Still, it is important to note that the characters assigned to `ordering` are interpreted as *factors*, whereas the characters assigned to `outcome` are interpreted as *factor values*. This difference may require the specification of both `ordering` and `outcome`, in particular, when only specific values of the factors in the ordering are possible outcomes. To illustrate, compare the following two function calls:

```
R> cna(d.pban, ordering = "T, PB", con = .75, cov = .75)
R> cna(d.pban, outcome = c("T=2", "PB=1"), ordering = "T, PB",
+     con = .75, cov = .75)
```

[10] The function `csf()` used in the following code builds the *csf* from a `cna()` solution object; see section 5.1.

The first call entails that any values of the factors T and PB, in that order, are located at the downstream end of the causal structure generating the data `d.pban`. It returns various solutions for PB=1 as well as for both T=0 and T=2. The second call, by contrast, narrows the search down to T=2 as only potential outcome value of factor T, such that no solutions for T=0 are produced.

A causal ordering excludes *all* values of a factor as potential causes of an outcome. However, a user might only wish to exclude *some* values as potential causes. This selective exclusion can be specified in the `exclude` argument. It is assigned a vector of character strings, where factor values to be excluded are listed to the left of the `"->"` sign, and the corresponding outcomes are listed to the right. For example, `exclude = "A=1,C=3 -> B=1"` excludes that the value 1 of factor A and the value 3 of factor C are considered as causes of the value 1 of factor B. It is also possible to exclude factor values as potential causes of multiple outcomes; for instance, `exclude = c("A,c -> B", "b,H -> D")`. In the context of *cs* and *fs* data, upper case letters are interpreted as 1, while lower case letters are interpreted as 0. If factor names have multiple letters, any upper case letter is interpreted as 1, and the absence of upper case letters as 0. For *mv* data, the "Factor=value" notation is required. The `exclude` argument can be used either independently or in conjunction with `outcome` and `ordering`. But if the assignments to `outcome` and `ordering` contradict those to `exclude`, the assignments to `exclude` will be disregarded.

For example, the following call narrows down the search beyond what is specified in the `outcome` and `ordering` arguments by excluding the causal relevance of C=2 for T=2 and of T=1 and V=0 for PB=1:

```
R> cna(d.pban, outcome = c("T=2", "PB=1"), ordering = "T, PB",
+     con = .75, cov = .75, exclude = c("C=2 -> T=2", "T=1,V=0 -> PB=1"))
```

Selective exclusion of certain causal relationships might also be the only type of prior knowledge available to an analyst. In that case, `exclude` can be used as sole search space constraint.

```
R> cna(d.jobsecurity, con = .85, cov = .85, exclude = c("s,c -> JSR",
+     "jsr, L -> R"))
```

In general, `cna()` should be given all the causal information about the interplay of the factors in the data that is available prior to the analysis. There often exist many MINUS-formulas that fit the data equally well. The more prior information `cna()` has at its disposal, the more specific the output will be, on average.

3.4. Maxstep

As will be exhibited in more detail in section 4, `cna()` builds atomic solution formulas (*asf*), *viz.* minimally necessary disjunctions of minimally sufficient conditions (*msc*), from the bottom up by gradually permuting and testing conjunctions and disjunctions of increasing complexity for sufficiency and necessity. The combinatorial search space that this algorithm has to scan depends on a variety of different aspects, for instance, on the number of factors in `x`, on the number of values these factors can take, on the number and length of the recovered *msc*, etc. As the search space may be too large to be exhaustively scanned in reasonable time, the argument `maxstep` allows for setting an upper bound for the complexity of the generated

asf. `maxstep` takes a vector of three integers $c(i, j, k)$ as input, entailing that the generated *asf* have maximally j disjuncts with maximally i conjuncts each and a total of maximally k factor values. The default is `maxstep = c(3,4,10)`. The user can set it to any complexity level if computational time and resources are not an issue.

The `maxstep` argument is particularly relevant for the analysis of high dimensional data and data featuring severe model ambiguities. As an example of the first kind, consider the data `d.highdim` comprising 50 crisp-set factors, V1 to V50, and 1191 cases, which were simulated from a presupposed data-generating structure with the outcomes $V13$ and $V11$ (see the **cna** reference manual for details). These data feature 20% noise and massive fragmentation. At the default `maxstep`, the following analysis, which finds the complete data-generating structure, takes a few seconds to complete; lowering `maxstep` to `c(2,3,10)` reduces that time to less than one second, at the expense of only finding half of the data-generating structure:

```
R> cna(d.highdim, outcome = c("V13", "V11"), con = .8, cov = .8)
R> cna(d.highdim, outcome = c("V13", "V11"), con = .8, cov = .8,
+     maxstep = c(2,3,10))
```

A telling example of extensive model ambiguities is the data set `d.volatile`. When only constrained by an ordering, `cna()` quickly recovers 416 complex solution formulas (*csf*) at the default `maxstep`. But those are far from all *csf* that fit `d.volatile` equally well. When `maxstep` is increased only slightly to `c(4,4,10)`, the number of *csf* jumps to 2860:[11]

```
R> cna(d.volatile, ordering = "VO2", maxstep = c(3,4,10))
R> vol1 <- cna(d.volatile, ordering = "VO2", maxstep = c(4,4,10))
R> csf(vol1, n.init = 3000)
```

If `maxstep` is further increased, the number of solutions explodes and the analysis soon fails to terminate in reasonable time. When a complete analysis cannot be completed, `cna()` can be told to only search for *msc* by setting the argument `suff.only` to its non-default value `TRUE`. As the search for *msc* is the part of a CNA analysis that is least computationally demanding, it will typically terminate quickly and, thus, shed some light on the dependencies among the factors in `x` even when the construction of all models is infeasible.

```
R> cna(d.volatile, ordering = "VO2", maxstep = c(8,10,40), suff.only = TRUE)
```

If `suff.only` is set to `TRUE`, CNA can process data of higher dimensionality than at the argument's default value. Yakovchenko *et al.* (2020), for example, run `cna()` on data comprising 73 exogenous factors with `suff.only = TRUE`. Based on the resulting *msc*, they then select a proper subset of those factors for further processing.

While the `maxstep` argument is valuable for controlling the search space in case of high-dimensional and ambiguous data, it also comes with a pitfall: it may happen that `cna()` fails to find a model because of a `maxstep` that is too low. An example is `d.jobsecurity`. At the default `maxstep`, `cna()` does not build a solution, but if `maxstep` is increased, two solutions are found.

[11] In the standard print method of `cna()`, the `n.init` parameter in `csf()` is set to 1000; to get all *csf*, this parameter needs to be increased. See section 5.1 for details.

```
R> ana.jsc.1 <- cna(d.jobsecurity, ordering = "JSR", con = .9, cov = .85)
R> csf(ana.jsc.1)[printCols]

[1] condition con       cov
<0 rows> (or 0-length row.names)

R> ana.jsc.2 <- cna(d.jobsecurity, ordering = "JSR", con = .9, cov = .85,
+    maxstep = c(3,5,12))
R> csf(ana.jsc.2)[printCols]

  condition                                    con   cov
1 S*V + C*l + L*P + R*V + S*c*R <-> JSR   0.906 0.859
2 C*l + R*V + P*v + S*c*R + S*C*P <-> JSR 0.912 0.853
Measures:
  con: standard consistency
  cov: standard coverage
```

In sum, there are two possible reasons why `cna()` fails to build a solution: (i) the chosen `maxstep` is too low; (ii) the chosen `con` and/or `cov` values are too high, meaning the processed data `x` are too noisy. Accordingly, in case of a null result, two paths should be explored (in that order): (i) gradually increase `maxstep`; (ii) lower `con` and `cov`, as described in section 3.2 above.

3.5. Negated outcomes

In classical logic, the law of Contraposition ensures that an expression of type $\Phi \leftrightarrow Y$ is equivalent to the expression that results from negating both sides of the double arrow: $\neg\Phi \leftrightarrow \neg Y$. Applied to the context of configurational causal modeling that entails that an *asf* for Y can be transformed into an *asf* for the negation of Y, *viz.* y, based on logical principles alone, i.e. without a separate data analysis. However, that transformability only holds for *asf* with perfect consistency and coverage (`con = cov = 1`) that are inferred from exhaustive (non-fragmented) data (see section 5.2 for details on exhaustiveness). If an *asf* of an outcome Y does not reach perfect consistency or coverage or is inferred from fragmented data, identifying the causes of y requires a separate application of `cna()`.

There are two ways to search for the causes of negated outcomes. The first is by simply specifying the factor values of interest in the `outcome` argument. While `outcome = c("A", "B")` yields MINUS-formulas for the positive outcomes A and B, `outcome = c("a", "b")` induces `cna()` to search for models of the corresponding negated outcomes. Alternatively, the argument `notcols` allows for negating the values of factors in *cs* and *fs* data (in case of *mv* data, `cna()` automatically searches for models of all possible values of endogenous factors, thereby rendering `notcols` redundant). If `notcols = "all"`, all factors are negated, i.e. their values i are replaced by $1 - i$. If `notcols` is given a character vector of factors in the data, only the values of the factors in that vector are negated. For example, `notcols = c("A", "B")` determines that only A and B are negated.

When processing *cs* or *fs* data, CNA should first be used to model the positive outcomes. If resulting *asf* and *csf* do not reach perfect consistency, coverage, and exhaustiveness scores

(and the causes of the negated outcomes are of interest), a second CNA should be run that negates the values of all factors that have been modeled as outcomes in the first CNA. To illustrate, we revisit our analysis of `d.autonomy` from section 3.3, which identified *AU* and *EM* as outcomes. The following two calls conduct analyses of the corresponding negated outcomes that produce the same solutions.

```
R> ana.aut.3 <- cna(dat.aut.1, outcome = c("au", "em"), con = .88, cov = .82)
R> csf(ana.aut.3)[printCols]

  condition                     con   cov
1 (sp <-> au)*(sp*co <-> em) 0.882 0.821
Measures:
  con: standard consistency
  cov: standard coverage

R> ana.aut.4 <- cna(dat.aut.1, ordering = "AU", con = .88, cov = .82,
+     notcols = c("AU", "EM"))
R> csf(ana.aut.4)[printCols]

  condition                     con   cov
1 (sp <-> au)*(sp*co <-> em) 0.882 0.821
Measures:
  con: standard consistency
  cov: standard coverage
```

4. The CNA algorithm

This section explains the working of the algorithm implemented in the `cna()` function. We first provide an informal summary and then a detailed outline in four stages. The aim of `cna()` is to find all *msc*, *asf*, and *csf* in the input data `x` that meet the thresholds `con` and `cov` of the sufficiency and necessity measures specified in `measures` in accordance with `outcome`, `ordering`, `exclude`, and `maxstep`. The algorithm starts with single factor values and tests whether they meet `con`; if that is not the case, it proceeds to test conjunctions of two factor values, then to conjunctions of three, and so on. Whenever a conjunction meets `con` (and no proper part of it has previously been identified to meet `con`), it is automatically a minimally sufficient condition *msc*, and supersets of it do not need to be tested any more. Then, it tests whether single *msc* meet `con` and `cov`; if not, it proceeds to disjunctions of two, then to disjunctions of three, and so on. Whenever a disjunction meets `con` and `cov` (and no proper part of it has previously been identified to meet `con` and `cov`), it is automatically a minimally necessary disjunction of *msc*, and supersets of it do not need to be tested any more. All and only those disjunctions of *msc* that meet both `con` and `cov` are then issued as *asf*. Finally, recovered *asf* are conjunctively concatenated to *csf* while ensuring that concatenation does not introduce new redundancies and that the resulting *csf* comply with the chosen tuning settings.

The `cna()` algorithm can be more specifically broken down into four stages.

Stage 1 On the basis of `outcome`, `ordering`, and `exclude`, `cna()` first builds a set of possible outcomes $\mathbf{O} = \{O_h{=}\omega_f, \ldots, O_m{=}\omega_g\}$ from the set of factors $\mathbf{F} = \{O_1, \ldots, O_n\}$ in `x`,[12] where $1 \leq h \leq m \leq n$, and second assigns a set of potential cause factors $\mathbf{C}_{O_i}$ from $\mathbf{F} \setminus \{O_i\}$ to every element $O_i{=}\omega_k$ of $\mathbf{O}$. If no `outcome` and `ordering` are provided, all value assignments to all elements of $\mathbf{F}$ are treated as possible outcomes in case of *mv* data, whereas in case of *cs* and *fs* data $\mathbf{O}$ is set to $\{O_1{=}1, \ldots, O_n{=}1\}$. If both `ordering` and `exclude` are empty, all values of all factors in $\mathbf{F} \setminus \{O_i\}$ are treated as possible causes of $O_i{=}\omega_k$, for every $O_i{=}\omega_k \in \mathbf{O}$.

Stage 2 `cna()` attempts to build a set $\mathbf{msc}_{\mathbf{O}_i=\omega_k}$ of minimally sufficient conditions that meet `con` for each $O_i{=}\omega_k \in \mathbf{O}$. To this end, it first checks for each value assignment $X_h{=}\chi_j$ of each element of $\mathbf{C}_{O_i}$, such that $X_h{=}\chi_j$ has a membership score above 0.5 in at least one case in `x`, whether $X_h{=}\chi_j \rightarrow O_i{=}\omega_k$ meets `con`. If, and only if, that is the case, $X_h{=}\chi_j$ is put into the set $\mathbf{msc}_{\mathbf{O}_i=\omega_k}$. Next, `cna()` checks for each conjunction of two factor values $X_m{=}\chi_j * X_n{=}\chi_l$ from $\mathbf{C}_{O_i}$, such that $X_m{=}\chi_j * X_n{=}\chi_l$ has a membership score above 0.5 in at least one case in `x` and no part of $X_m{=}\chi_j * X_n{=}\chi_l$ is already contained in $\mathbf{msc}_{\mathbf{O}_i=\omega_k}$, whether $X_m{=}\chi_j * X_n{=}\chi_l \rightarrow O_i{=}\omega_k$ meets `con`. If, and only if, that is the case, $X_m{=}\chi_j * X_n{=}\chi_l$ is put into the set $\mathbf{msc}_{\mathbf{O}_i=\omega_k}$. Next, conjunctions of three factor values with no parts already contained in $\mathbf{msc}_{\mathbf{O}_i=\omega_k}$ are tested, then conjunctions of four factor values, etc., until either all logically possible conjunctions of the elements of $\mathbf{C}_{O_i}$ have been tested or `maxstep` is reached. Every non-empty $\mathbf{msc}_{\mathbf{O}_i=\omega_k}$ is passed on to the third stage.

Stage 3 `cna()` attempts to build a set $\mathbf{asf}_{\mathbf{O}_i=\omega_k}$ of atomic solution formulas for every $O_i{=}\omega_k \in \mathbf{O}$, which has a non-empty $\mathbf{msc}_{\mathbf{O}_i=\omega_k}$, by disjunctively concatenating the elements of $\mathbf{msc}_{\mathbf{O}_i=\omega_k}$ to minimally necessary conditions of $O_i{=}\omega_k$ that meet `con` and `cov`. To this end, it first checks for each single condition $\Phi_h \in \mathbf{msc}_{O_i=\omega_k}$ whether $\Phi_h \rightarrow O_i{=}\omega_k$ meets `con` and `cov`. If, and only if, that is the case, Φ_h is put into the set $\mathbf{asf}_{\mathbf{O}_i=\omega_k}$. Next, `cna()` checks for each disjunction of two conditions $\Phi_m + \Phi_n$ from $\mathbf{msc}_{O_i=\omega_k}$, such that no part of $\Phi_m + \Phi_n$ is already contained in $\mathbf{asf}_{\mathbf{O}_i=\omega_k}$, whether $\Phi_m + \Phi_n \rightarrow O_i{=}\omega_k$ meets `con` and `cov`. If, and only if, that is the case, $\Phi_m + \Phi_n$ is put into the set $\mathbf{asf}_{\mathbf{O}_i=\omega_k}$. Next, disjunctions of three conditions from $\mathbf{msc}_{O_i=\omega_k}$ with no parts already contained in $\mathbf{asf}_{\mathbf{O}_i=\omega_k}$ are tested, then disjunctions of four conditions, etc., until either all logically possible disjunctions of the elements of $\mathbf{msc}_{O_i=\omega_k}$ have been tested or `maxstep` is reached. Every non-empty $\mathbf{asf}_{\mathbf{O}_i=\omega_k}$ is passed on to the fourth stage.

Stage 4 `cna()` calls the function `csf()`, which builds a set $\mathbf{csf}_{\mathbf{O}}$ of complex solution formulas. This is done in a stepwise manner as follows. First, all logically possible conjunctions of exactly one element from every non-empty $\mathbf{asf}_{O_i=\omega_k}$ are constructed. Second, the conjunctions resulting from step 1 are freed of structural redundancies (cf. p. 8 above), and tautologous and contradictory solutions as well as solutions with constant factors are eliminated. Third, *csf* with so-called *partial structural redundancies*, which may arise from noisy data, are eliminated (see the Appendix on p. 50 for more). Fourth, if `acyclic.only = TRUE`, solutions with cyclic substructures are removed (see section 5.4). Fifth, if `func.minimal.only = TRUE`, solutions that are not functionally minimal are

[12] Note that if `x` is a data frame, `cna()` first transforms `x` into a configuration table by means of `configTable(x)`.

removed (see section 5.5). Sixth, for those solutions that were modified in the previous steps, the sufficiency and necessity scores specified by `measures` are re-calculated and solutions that no longer reach `con` or `cov` are deleted. Finally, the remaining solutions are checked for submodel relations: if one of them is a submodel of another one, it is deleted—the output of `cna()` shall be maximally informative. The remaining solutions are then returned as $\mathbf{csf_O}$. If there is only one non-empty set $\mathbf{asf}_{\mathrm{O}_i=\omega_k}$, $\mathbf{csf_O}$ is identical to $\mathbf{asf}_{\mathrm{O}_i=\omega_k}$.

To illustrate, the following code chunk, first, simulates the data in Table 3c, p. 12, and second, runs `cna()` (and `csf()`) on that data at `con = .8` and `cov = .8`, with the default `measures` and `maxstep`, and without specification of `outcome`, `ordering`, and `exclude`.

```
R> dat5 <- allCombs(c(2, 2, 2, 2, 2)) -1
R> dat6 <- selectCases("(A + B <-> C)*(A*B + D <-> E)", dat5)
R> set.seed(3)
R> tab3c <- makeFuzzy(dat6, fuzzvalues = seq(0, 0.4, 0.01))
R> cna(tab3c, con = .8, cov = .8)
```

Table 3c contains data of type *fs*, meaning that the values in the data matrix are interpreted as membership scores in fuzzy sets. As is customary for this data type, we use uppercase italics for membership in a set and lowercase italics for non-membership. In the absence of any prior causal knowledge, the set of possible outcomes is determined to be $\mathbf{O} = \{A, B, C, D, E\}$ in stage 1, that is, the presence of each factor in Table 3c is treated as a possible outcome. Moreover, all other factors are potential cause factors of every element of $\mathbf{O}$, hence, $\mathbf{C}_A = \{\mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E}\}$, $\mathbf{C}_B = \{\mathrm{A}, \mathrm{C}, \mathrm{D}, \mathrm{E}\}$, $\mathbf{C}_C = \{\mathrm{A}, \mathrm{B}, \mathrm{D}, \mathrm{E}\}$, $\mathbf{C}_D = \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{E}\}$, and $\mathbf{C}_E = \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}\}$.

In stage 2, `cna()` succeeds in building non-empty sets of minimally sufficient conditions in compliance with `con` and `con` for all elements of $\mathbf{O}$: $\mathbf{msc}_A = \{B{*}d{*}E\}$, $\mathbf{msc}_B = \{C{*}d,\ d{*}E,\ a{*}C{*}D,\ a{*}C{*}E,\ a{*}C{*}e\}$, $\mathbf{msc}_C = \{A,\ B,\ d{*}E\}$, $\mathbf{msc}_D = \{b{*}E,\ a{*}E,\ c{*}E\}$, $\mathbf{msc}_E = \{D,\ A{*}B,\ A{*}C\}$. But only the elements of $\mathbf{msc}_C$ and $\mathbf{msc}_E$ can be disjunctively combined to atomic solution formulas that meet `cov` in stage 3: $\mathbf{asf}_C = \{A + B \leftrightarrow C\}$ and $\mathbf{asf}_E = \{D + A{*}B \leftrightarrow E,\ D + A{*}C \leftrightarrow E\}$. For the other three factors in $\mathbf{O}$, the `cov` threshold of 0.8 cannot be satisfied. `cna()` therefore abstains from issuing *asf* for A, B and D.

Finally, in stage 4 one redundancy-free *csf* is built from the inventory of *asf* in $\mathbf{asf}_C$ and $\mathbf{asf}_E$, which constitutes `cna()`'s final output for Table 3c:

$$(A\ +\ B \leftrightarrow C) * (D\ +\ A{*}B \leftrightarrow E) \qquad con = 0.836;\ cov = 0.897 \tag{3}$$

5. The output of CNA

5.1. Customizing the output

The default output of `cna()` first lists the provided ordering (if any), second, the pre-identified outcomes (if any), third, the implemented sufficiency and necessity measures, fourth, the recovered *asf*, and fifth, the *csf*. *Asf* and *csf* are ordered by complexity and the product of

their `con` and `cov` scores. For *asf*, three attributes are standardly computed: `con`, `cov`, and `complexity`; *csf* standardly feature these three attributes plus `cyclic` and `fm_empirical`. `con` and `cov` correspond to a solution's scores on the selected variants of consistency and coverage (see section 3.2 above), `complexity` expresses the number of factor value appearances on the left-hand sides of "→" or "↔" in *asf* and *csf*, `cyclic` indicates whether a solution contains a cyclic substructure, and `fm_empirical` whether the solution is functionally minimal (in the analyzed data).

As mentioned on page 20, `cna()` can also return the scores on the evaluation measures not used for model building by requesting them via the argument `details`. A number of additional solution attributes, all of which will be explained below, can be computed: `exhaustiveness` and `faithfulness` for both *asf* and *csf*, as well as `coherence` for *csf*. The attributes to be computed are specified by assigning `details` a character vector: for example, `details = c("faithfulness", "exhaustiveness")`—the strings can be abbreviated, e.g. `"fa"` for `"faithfulness"`, `"e"` for `"exhaustiveness"`, etc.

```
R> cna(d.educate, details = c("e", "fa", "co", "cy", "PAcon", "PACcov"))
```

The output of `cna()` can be further customized through the argument `what` that controls which solution items to print. It can be given a character string specifying the requested solution items: `"t"` stands for the configuration table, `"m"` for minimally sufficient conditions (*msc*), `"a"` for *asf*, `"c"` for *csf*, and `"all"` for all solution items.

```
R> cna(d.educate, what = "tm")
R> cna(d.educate, what = "mac")
R> cna(d.educate, what = "all")
```

As shown in section 3.4, it can happen that many *asf* and *csf* fit the data equally well. The standard output of `cna()` only features 5 solution items of each type. To recover all *msc* and *asf* the functions `msc(x)` and `asf(x)` are available, where `x` is a solution object of `cna()`.

```
R> vol2 <- cna(d.volatile, ordering = "VO2", con = .9, cov = .9)
R> msc(vol2)
R> asf(vol2)
R> print(asf(vol2), Inf)
```

While `msc()` and `asf()` simply access the complete sets of *msc* and *asf* stored in `x`, the *csf* are not stored in `x`. The construction of *csf* in the fourth stage of the CNA algorithm is not conducted by the `cna()` function itself, rather, it is outsourced to the function `csf()` with these main arguments:

```
csf(x, n.init = 1000, acyclic.only = x$acyclic.only,
   details = x$details, cycle.type = x$cycle.type,
   func.minimal.only = x$control$func.minimal.only, verbose = FALSE)
```

The argument `details` is the same as in `cna()` (see p. 20); the arguments `acyclic.only` and `cycle.type` will be further discussed in section 5.4, `func.minimal.only` in section 5.5, `n.init` and `verbose` are explained in the remainder of this one. It can happen that the set $\mathbf{asf}_{\mathbf{O}_i=\omega_k}$ contains too many *asf* to construct all *csf* in reasonable time. The argument `n.init`

therefore allows for controlling how many conjunctions of *asf* are initially built in the first step of *csf* construction (see stage 4 of the CNA algorithm); it defaults to 1000. Increasing or lowering that default results in more or less *csf* being built and in longer or shorter computing times, respectively.

```
R> csf(vol2, n.init = 2000)
R> csf(vol2, n.init = 100)
```

Setting the argument `verbose` to its non-default value `TRUE` prints some information about the *csf* construction process to the console, e.g. how many structural redundancies or cyclic substructures have been eliminated along the way.

```
R> csf(vol2, verbose = TRUE)
```

5.2. Exhaustiveness and faithfulness

Exhaustiveness and faithfulness are two measures of model fit that quantify the degree of correspondence between the configurations that are, in principle, compatible with a solution $\mathbf{m}$ and the configurations actually contained in the data from which $\mathbf{m}$ is derived. To demonstrate those measures, let $\mathbf{F_m}$ symbolize the set of factors with values contained in $\mathbf{m}$. Exhaustiveness is high when *all* or most configurations of the factors in $\mathbf{F_m}$ that are *compatible* with $\mathbf{m}$ are actually contained in the data. More specifically, it amounts to the ratio of the number of configurations over $\mathbf{F_m}$ in the data that are compatible with $\mathbf{m}$ to the total number of configurations over $\mathbf{F_m}$ that are compatible with $\mathbf{m}$. To illustrate, consider `d.educate`, which contains all configurations that are compatible with the two *csf* issued by `cna()` and `csf()`:

```
R> printCols <- c("condition", "con", "cov", "exhaustiveness")
R> csf(cna(d.educate, details = "exhaust"))[printCols]
```

```
  condition                        con cov exhaustiveness
1 (L + G <-> E)*(U + D <-> L)        1   1              1
2 (U + D + G <-> E)*(U + D <-> L)    1   1              1
Measures:
  con: standard consistency
  cov: standard coverage
```

If, say, the first configuration in `d.educate` (*viz.* $U{*}D{*}L{*}G{*}E$) is not observed or removed—as in `d.educate[-1,]`—`cna()` still builds the same solutions (with perfect consistency and coverage). In that case, however, the resulting *csf* are not exhaustively represented in the data, for one configuration that is compatible with both *csf* is not contained therein.

```
R> csf(cna(d.educate[-1,], details = "exhaust"))[printCols]
```

```
  condition                        con cov exhaustiveness
1 (L + G <-> E)*(U + D <-> L)        1   1          0.875
2 (U + D + G <-> E)*(U + D <-> L)    1   1          0.875
```

```
Measures:
  con: standard consistency
  cov: standard coverage
```

In a sense, faithfulness is the complement of exhaustiveness. It is high when *no* or only few configurations of the factors in $\mathbf{F_m}$ that are *incompatible* with $\mathbf{m}$ are in the data. More specifically, faithfulness amounts to the ratio of the number of configurations over $\mathbf{F_m}$ in the data that are compatible with $\mathbf{m}$ to the total number of configurations over $\mathbf{F_m}$ in the data. The two *csf* resulting from `d.educate` also reach perfect faithfulness:

```
R> printCols <- c("condition", "con", "cov", "faithfulness")
R> csf(cna(d.educate, details = "faithful"))[printCols]

  condition                        con cov faithfulness
1 (L + G <-> E)*(U + D <-> L)        1   1            1
2 (U + D + G <-> E)*(U + D <-> L)    1   1            1
Measures:
  con: standard consistency
  cov: standard coverage
```

If we add a configuration that is not compatible with these *csf*, say, $U{*}D{*}l{*}G{*}e$ and lower the consistency threshold, the same solutions along with one other result—this time, however, with non-perfect faithfulness scores.

```
R> csf(cna(rbind(d.educate, c(1,1,0,1,0)), con = .8,
+          details = "fa"))[printCols]

  condition                          con cov faithfulness
1 (L + G <-> E)*(U + D <-> L)      0.857   1        0.889
2 (U + D + G <-> E)*(E <-> L)      0.857   1        0.778
3 (U + D + G <-> E)*(U + D <-> L) 0.857   1        0.889
Measures:
  con: standard consistency
  cov: standard coverage
```

If both exhaustiveness and faithfulness are high, the configurations over $\mathbf{F_m}$ in the data are all and only the configurations of the factors in $\mathbf{F_m}$ that are compatible with $\mathbf{m}$. Low exhaustiveness and/or faithfulness, by contrast, means that the data do not contain many configurations of the factors in $\mathbf{F_m}$ compatible with $\mathbf{m}$ and/or the data contain many configurations not compatible with $\mathbf{m}$. In general, solutions with higher exhaustiveness and faithfulness scores are preferable to solutions with lower scores.

5.3. Coherence

Coherence is a measure of model fit that is custom-built for *csf*. It measures the degree to which the *asf* combined in a *csf* cohere, that is, are instantiated together in the data rather than independently of one another. Coherence is intended to capture the following intuition.

Suppose a *csf* entails that A is a sufficient cause of B, which, in turn, is entailed to be a sufficient cause of C. Corresponding data δ should be such that the $A-B$ link of that causal chain and the $B-C$ link are either both instantiated or both not instantiated in the cases recorded in δ. By contrast, a case in δ such that, say, only the $A-B$ link is instantiated but the $B-C$ link is not, pulls down the coherence of that *csf*. The more such non-cohering cases are contained in δ, the lower the overall coherence score of the *csf*.

Coherence is more specifically defined as the ratio of the number of cases satisfying all *asf* contained in a *csf* to the number of cases satisfying at least one *asf* in the *csf*. More formally, let a *csf* contain $asf_1, asf_2, \ldots, asf_n$; coherence then amounts to (where $|\ldots|_\delta$ represents the cardinality of the set of cases in δ satisfying the corresponding expression):

$$\frac{|\, asf_1 * asf_2 * \ldots * asf_n \,|_\delta}{|\, asf_1 + asf_2 + \ldots + asf_n \,|_\delta}$$

To illustrate, we add a case of type $U*d*L*g*e$ to `d.educate`. When applied to the resulting data (`d.edu.exp1`), `cna()` and `csf()` issue two *csf*.

```
R> d.edu.exp1 <- rbind(d.educate, c(1,0,1,0,0))
R> printCols <- c("condition", "con", "cov", "coherence")
R> csf(cna(d.edu.exp1, con = .8, details = "cohere"))[printCols]

  condition                          con cov coherence
1 (L + G <-> E)*(U + D <-> L)     0.875   1     0.889
2 (U + D + G <-> E)*(U + D <-> L) 0.875   1     0.889
Measures:
  con: standard consistency
  cov: standard coverage
```

In the added case, none of these two *csf* cohere, as only one of their component *asf* is satisfied.

Coherence is an additional parameter of model fit that allows for selecting among multiple solutions: the higher the coherence score of a *csf*, the better the overall model fit.

5.4. Cycles

Detecting causal cycles is one of the most challenging tasks in causal data analysis—in all methodological traditions. One reason is that factors in a cyclic structure are so highly interdependent that, even under optimal discovery conditions, the diversity of (observational) data tends to be too limited to draw informative conclusions about the data-generating structure. Various methods in fact are restricted to analyzing acyclic structures only (most notably, Bayesian network methods, cf. Spirtes *et al.* 2000).

`cna()` and `csf()` output cyclic *csf* if they fit the data. The solution attribute `cyclic` identifies those *csf* that contain a cyclic substructure. A causal structure has a cyclic substructure if, and only if, it contains a directed causal path from at least one cause back to itself. The MINUS theory spells this criterion out more explicitly as follows:

Cycle A complex MINUS-formula **m** has a cyclic substructure if, and only if, **m** contains a sequence $\langle Z_1, Z_2, \ldots, Z_n \rangle$ such that every Z_i is contained in an atomic MINUS-formula of Z_{i+1} and $Z_1 = Z_n$ in **m**.

To illustrate, consider the following analysis of the `d.autonomy` data.

```
R> printCols <- c("condition", "con", "cov", "cyclic")
R> csf(cna(d.autonomy, ordering = "AU", con = .9, cov = .94,
+     details = "cy", maxstep = c(2, 2, 8)))[printCols]
```

```
  condition                                        con   cov cyclic
1 (SP*RE + CO*cn <-> EM)*(ci + EM*co <-> SP) 0.938 0.947   TRUE
2 (SP*co + CO*cn <-> EM)*(ci + EM*co <-> SP) 0.938 0.947   TRUE
3 (SP*RE + CO*cn <-> EM)*(ci + EM*RE <-> SP) 0.928 0.947   TRUE
4 (SP*co + CO*cn <-> EM)*(ci + EM*RE <-> SP) 0.928 0.947   TRUE
Measures:
  con: standard consistency
  cov: standard coverage
```

All *csf* inferred in this analysis contain the cyclic sequence $\langle SP, EM, SP\rangle$ and, thus, represent causal cycles. Typically, when cyclic models fit the data, the output of `cna()` and `csf()` is very ambiguous. Therefore, if there are independent reasons to assume that the data are not generated by a cyclic structure, both `cna()` and `csf()` have the argument `acyclic.only`, which, if set to its non-default value `TRUE`, prevents solutions with cycles from being returned and, thereby, reduces model ambiguities. For example, by switching `acyclic.only` from `FALSE` to `TRUE` in the following analysis, the solution space is reduced from 72 to 23:

```
R> csf(cna(d.irrigate, con = .75, cov = .75, acyclic.only = F)) |> nrow()
```

```
[1] 72
```

```
R> csf(cna(d.irrigate, con = .75, cov = .75, acyclic.only = T)) |> nrow()
```

```
[1] 23
```

The `cycle.type` argument—also available in both `cna()` and `csf()`—controls whether a cyclic sequence $\langle Z_1, Z_2, \ldots, Z_n\rangle$ is composed of factors (`cycle.type = "factor"`), which is the default, or factor values (`cycle.type = "value"`). To illustrate the difference, if `cycle.type = "factor"`, (4) counts as cyclic:

$$(A + B \leftrightarrow C) * (c + D \leftrightarrow A) \tag{4}$$

The factor A (with value 1) appears in an *asf* of C (i.e. C=1), and the factor C (with value 0) appears in an *asf* of A. But if `cycle.type = "value"`, (4) does not pass as cyclic. Although A appears in an *asf* of C=1, that same value of C does not appear in an *asf* of A; rather, C=0 appears in the *asf* of A.

The function behind the solution attribute `cyclic` and behind the corresponding `cna()` arguments is also available as stand-alone function `cyclic()` (see the **cna** reference manual for details).

5.5. Functional minimality

The functional minimality constraint introduced in section 2.3 is operationalized by the argument `func.minimal.only`, which defaults to `FALSE`: a *csf* counts as functionally minimal if no other *csf* in the output accounts for the same outcome(s) on the basis of no more factors, and for at least one of them on the basis of strictly fewer factors. Whether a *csf* satisfies that criterion is indicated by the standard solution attribute `fm_empirical` (section 5.1). Setting `func.minimal.only = TRUE` removes all *csf* with `fm_empirical = FALSE` in the fourth stage of the CNA algorithm (see p. 27).

As an illustration, consider the air-conditioning scenario by means of which Zhang and Zhang (2023) motivate the functional minimality constraint. Billy's office is cooled by a central air conditioning system and a window air conditioner, Cathy's office by the same central system and a dehumidifier. Due to a construction flaw, the operation of Billy's window air conditioner raises the humidity in Cathy's office, which causes her discomfort even when the central air conditioning is on. Her dehumidifier remedies this. Billy feels comfortable (E1) if, and only if, the central air conditioning (A) or his window air conditioner (B) is on. Cathy feels comfortable (E2) if, and only if, the central air conditioning is on and her dehumidifier (C) is on, or the central air conditioning is on and Billy's window air conditioner is off. Here is the complex MINUS-formula representing that structure:

$$(A + B \leftrightarrow E1) * (A{*}b + A{*}C \leftrightarrow E2) \tag{5}$$

The ideal data corresponding to that structure are generated and analyzed as follows:

```
R> printCols <- c("condition", "con", "cov", "cyclic", "fm_empirical")
R> aircon <- selectCases("(A + B <-> E1)*(A*b + A*C <-> E2)")
R> csf(cna(aircon, outcome = c("E1", "E2")))[printCols]

  condition                          con cov cyclic fm_empirical
1 (A + B <-> E1)*(A*b + A*C <-> E2)    1   1  FALSE         TRUE
2 (A + B <-> E1)*(A*C + b*E1 <-> E2)   1   1  FALSE        FALSE
3 (B + E2 <-> E1)*(A*b + A*C <-> E2)   1   1  FALSE         TRUE
4 (B + E2 <-> E1)*(A*C + b*E1 <-> E2)  1   1   TRUE        FALSE
Measures:
  con: standard consistency
  cov: standard coverage
```

Two of the four resulting *csf* correctly account for Cathy feeling comfortable (E2) by $A{*}b + A{*}C$, while the other two incorrectly express that outcome as $A{*}C + b{*}E1$. And indeed, the two former *csf* are flagged `fm_empirical = TRUE`, whereas the two latter are not functionally minimal because E1 is redundant in accounting for E2. Setting `func.minimal.only = TRUE` removes them (because all retained *csf* are then functionally minimal, the `fm_empirical` attribute is no longer attached to the output):

```
R> csf(cna(aircon, outcome = c("E1", "E2"),
+     func.minimal.only = TRUE))[setdiff(printCols, "fm_empirical")]
```

```
  condition                              con cov cyclic
1 (A + B <-> E1)*(A*b + A*C <-> E2)    1   1  FALSE
2 (B + E2 <-> E1)*(A*b + A*C <-> E2)   1   1  FALSE
Measures:
  con: standard consistency
  cov: standard coverage
```

In light of this result, Zhang and Zhang (2023) contend that imposing functional minimality helps to reduce model ambiguities, while retaining the data-generating causal structure.

The function behind the solution attribute `fm_empirical` is also available as a stand-alone function `fm_empirical(cond)` (see the **cna** reference manual for details). It is complemented by `fm_structural(cond, x = full.ct(cond))`. `fm_empirical()` applies that criterion within the vector of solution formulas `cond`, comparing each element to the other elements that have the same set of outcomes. `fm_structural()` checks every element of `cond` individually for redundant factors, not by comparison with the other elements of `cond` but by reference to the ideal data generated from that element. The two can disagree.[13] In the analysis above, both *asf* for E2 are `fm_structural = TRUE`, whereas only $A{*}b + A{*}C$ is `fm_empirical = TRUE`:

```
R> conds <- c("(A*b + A*C <-> E2)", "(A*C + b*E1 <-> E2)")
R> fm_structural(conds)

 (A*b+A*C<->E2) (A*C+b*E1<->E2)
           TRUE            TRUE

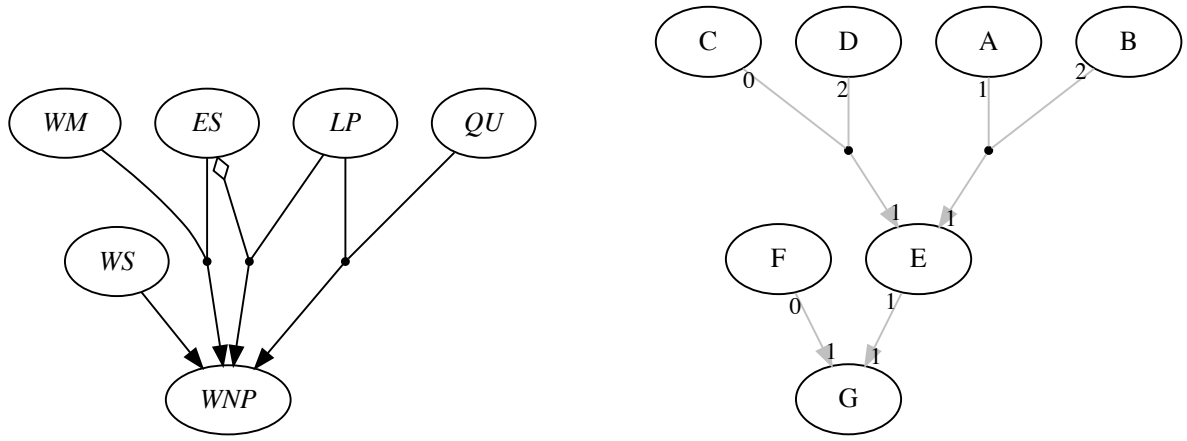
R> fm_empirical(conds)

 (A*b+A*C<->E2) (A*C+b*E1<->E2)
           TRUE           FALSE
```

5.6. Plotting the output

MINUS-formulas can be visualized as causal hypergraphs (Baumgartner and Falk forthcoming), which are related to directed acyclic graphs (DAGs; Greenland *et al.* 1999; Spirtes *et al.* 2000), the most widely used tool for visualizing causal structures. But while edges in DAGs connect exactly two nodes, indicating the direction of causation, edges in hypergraphs can connect more than two nodes and, thereby, represent more than just the direction of causation. Causal hypergraphs can merge nodes into bundles and then connect these bundles to other nodes. This allows for representing conjunctive and disjunctive groupings of causes and, accordingly, for capturing the causal complexity encoded in MINUS-formulas. Furthermore, while DAGs are assumed not to contain cycles, causal hypergraphs may include cycles to accommodate the fact that MINUS-formulas may have cyclic substructures.

[13] Zhang and Zhang (2023) formulate functional minimality for idealized structures only, that is, in the purely conceptual context of defining causal relevance. `fm_empirical()` and `fm_structural()` are two different operationalizations of that constraint for the context of causal discovery, approximating it for non-ideal empirical data and the solution sets with which CNA works.



(a) set-CHG (b) mv-CHG

Figure 3: Causal Hypergraphs.

For convenience, we use the acronym CHG to refer to causal hypergraphs. A CHG is a pair $(\mathbf{F}, \mathbf{E})$, where $\mathbf{F}$ is a set of nodes and $\mathbf{E}$ is a set of ordered pairs $\langle \mathbf{F}_i, \mathbf{F}_j \rangle$ of disjoint subsets of $\mathbf{F}$.[14] Each of these ordered pairs $\langle \mathbf{F}_i, \mathbf{F}_j \rangle \in \mathbf{E}$ is called a directed *hyperedge*. The subset $\mathbf{F}_i$ is called the *tail* of the hyperedge, $\mathbf{F}_j$ is its *head*. The heads of hyperedges in CHGs representing MINUS-formulas are always singleton sets, whereas their tails can contain multiple elements.[15] Just as edges in DAGs, hyperedges in CHGs represent the relation of direct causal relevance. But while nodes in DAGs represent factors or variables such as A and B, the nodes in CHGs represent factor values such as A, b, or C=3. Hence, DAGs represent causal relationships between factors or variables, whereas CHGs represent causal relationships between factor values.

There are two types of CHGs: *set-CHGs* for causal structures involving values of crisp-set or fuzzy-set factors and *mv-CHGs* for structures of multi-value factors. Besides nodes and directed hyperedges, set-CHGs contain two further graphical elements: "•" for bundling nodes in a conjunction and "◊" at tails of hyperedges for negating factor values. Mv-CHGs also symbolize conjunction through "•", but instead of a negation sign, they feature numeric values directly assigned to the tails and heads of hyperedges indicating the factor values that are connected by the hyperedge. In both set- and mv-CHGs, hyperedges with the same head form a disjunction. Figure 3a is an example of a set-CHG, and Figure 3b an example of an mv-CHG.

The R package **causalHyperGraph** (Falk *et al.* 2024) provides functions to visualize the output of `cna()` as CHGs. The most basic plotting function is `plot(x)`, which takes a solution object `x` of `cna()` as input and draws the solution formulas contained in `x` as CHGs. For example, the following code draws the set-CHG in Figure 3a from the solution object `cna()` infers from `d.women`:

```
R> library(causalHyperGraph)
R> ana.d.women <- cna(d.women)
```

[14] For more on hypergraphs see e.g. (Gallo *et al.* 1993) or (Bretto 2013).
[15] Such hyperedges are also called *backward* hyperedges, cf. (Gil-Pons *et al.* 2024).

```
R> plot(ana.d.women)
```

There is also a function `causalHyperGraph(x)` that takes a character vector `x` expressing MINUS-formulas as input and draws the corresponding CHGs. To illustrate, the mv-CHG in Figure 3b is drawn as follows:

```
R> causalHyperGraph("(A=1*B=2 + C=0*D=2 <-> E=1)*(E=1 + F=0 <-> G=1)")
```

6. Interpreting the output

The ultimate output of `cna()` and `csf()` is the set $\mathbf{csf_O}$ of *csf*, which may coincide with *asf* if the data contain only one endogenous factor. The causal inferences warranted by the data input `x`, relative to the selected `measures` and their associated `con` and `cov` thresholds, while adhering to the specified `outcome`, `ordering`, `exclude`, and `maxstep` settings, must be drawn from the set $\mathbf{csf_O}$. This section explains the final interpretative step in a CNA analysis.

There are three possible types of outputs:

1. $\mathbf{csf_O}$ contains no *csf* (and, correspondingly, no *asf*);
2. $\mathbf{csf_O}$ contains exactly one *csf* (and, correspondingly, exactly one *asf* for each endogenous factor);
3. $\mathbf{csf_O}$ contains more than one *csf* (and, correspondingly, more than one *asf* for at least one endogenous factor).

6.1. No solution

As indicated in section 3.4, a null result can have two sources: either the data are too noisy to render the `con` and `cov` thresholds satisfiable or the `maxstep` is too low. If increasing `maxstep` does not yield solutions at the chosen `con` and `cov` thresholds, the latter may be lowered, preferably with a concomitant robustness analysis as described in section 3.2. If no solutions are recovered at `con = cov = .7`, the data are too noisy to warrant reliable causal inferences. Users are then advised to go back to the data and follow standard guidelines (known from other methodological frameworks) to improve data quality, e.g. by integrating further relevant factors into the analysis, enhancing the control of unmeasured causes, expanding the population of cases or disregarding inhomogeneous cases, correcting for measurement error, supplying missing values, etc.

It must be emphasized again (see section 2.4) that, under normal circumstances, an empty $\mathbf{csf_O}$ does not warrant the conclusion that the factors contained in the data input `x` are causally irrelevant to one another. The inference to causal irrelevance is much more demanding than the inference to causal relevance. A null result only furnishes evidence for causal irrelevance if there are independent reasons to assume that all potentially relevant factors are measured in `x` and that `x` exhausts the space of empirically possible configurations.

6.2. A unique solution

That $\mathbf{csf_O}$ contains exactly one *csf* is the optimal completion of a CNA analysis. It means that the data input `x` contains sufficient evidence for a determinate causal inference. The

factor values on the left-hand sides of "↔" in the *asf* constituting that *csf* can be interpreted as causes of the factor values on the right-hand sides. Moreover, their conjunctive, disjunctive, and sequential groupings reflect the actual properties of the data-generating causal structure.

Plainly, as with any other method of causal inference, the reliability of CNA's causal conclusions essentially hinges on the quality of the processed data. If the data satisfy homogeneity (see section 2.4), a unique solution is guaranteed to correctly reflect the data-generating structure. With increasing data deficiencies (noise, fragmentation), the (inductive) risk of committing causal fallacies inevitably increases as well. For details on the degree to which the reliability of CNA's causal conclusions decreases with increasing data deficiencies see (Baumgartner and Ambühl 2020) and (Parkkinen and Baumgartner 2023).

6.3. Multiple solutions

If $\mathbf{csf_O}$ has more than one element, the processed data underdetermine their own causal modeling. That means the evidence contained in the data is insufficient to determine which of the solutions contained in $\mathbf{csf_O}$ corresponds to the data-generating causal structure. An output set of multiple solutions $\{csf_1, csf_2, ..., csf_n\}$ is to be interpreted *disjunctively*: the data-generating causal structure is

$$csf_1 \text{ OR } csf_2 \text{ OR } ... \text{ OR } csf_n$$

but, based on the evidence contained in the data, it is ambiguous which of these csf is actually operative.

That empirical data underdetermine their own causal modeling is a very common phenomenon in all methodological traditions (Simon 1954; Spirtes *et al.* 2000, 59-72; Kalisch *et al.* 2012; Eberhardt 2013; Baumgartner and Thiem 2017). But while some methods are designed to automatically generate all fitting models, e.g. Bayesian network methods and configurational comparative methods, other methods rely on search heuristics that zoom in on one best fitting model only, e.g. logic regression or regression analytic methods, more generally. Whereas model ambiguities have long been a thoroughly investigated topic in certain traditions, such as Bayesian network methods, they are only beginning to be studied in the literature on configurational comparative methods.

CNA—on a par with any other method—cannot disambiguate what is empirically underdetermined. Rather, it draws those and only those causal conclusions for which the data *de facto* contain evidence. In cases of empirical underdetermination it therefore renders transparent all data-fitting models and leaves the disambiguation up to the analyst.

That `cna()` and `csf()` issue multiple solutions for some data input `x` does not necessarily mean that `x` is deficient. In fact, even data that are *ideal* by all quality standards of configurational causal modeling can give rise to model ambiguities. The following simulates a case in point:

```
R> dat7 <- selectCases("a*B + A*b + B*C <-> D")
R> printCols <- c("condition", "con", "cov", "exhaustiveness","faithfulness")
R> csf(cna(dat7, details = c("ex", "fa")))[printCols]

  condition             con cov exhaustiveness faithfulness
1 a*B + A*b + A*C <-> D   1   1              1            1
2 a*B + A*b + B*C <-> D   1   1              1            1
```

```
Measures:
  con: standard consistency
  cov: standard coverage
```

`dat7` induces perfect consistency and coverage scores and is free of fragmentation; it contains all and only the configurations that are compatible with the target structure, which accordingly is exhaustively and faithfully reflected in `dat7`. Nonetheless, two models can be inferred. The causal structures expressed by these two models generate the exact same data, meaning they are *empirically indistinguishable*.

Although a unique solution is more determinate and, thus, preferable to multiple solutions, the fact that `cna()` and `csf()` generate multiple equally data-fitting models is not generally an uninformative result. In the above example, both models feature $a{*}B + A{*}b$. That is, the data contain enough evidence to establish the joint relevance of $a{*}B$ and of $A{*}b$ for D (on alternative paths). What is more, it can be conclusively inferred that D has a further complex cause, *viz.* either $A{*}C$ or $B{*}C$. It is merely an open question which of these candidate causes is actually operative.

That different model candidates have some *msc* in common is a frequent phenomenon. Here's a real-life example, where two alternative causes, *viz.* C=1 + F=2, are present in all solutions:

```
R> csf(cna(d.pban, cov = .95, maxstep = c(3, 5, 10)))["condition"]

  condition
1 C=1 + F=2 + C=0*F=1 + C=2*V=0 <-> PB=1
2 C=1 + F=2 + C=0*T=2 + C=2*V=0 <-> PB=1
3 C=1 + F=2 + C=2*F=0 + C=0*F=1 + F=1*V=0 <-> PB=1
4 C=1 + F=2 + C=2*F=0 + C=0*T=2 + F=1*V=0 <-> PB=1
5 C=1 + F=2 + C=0*F=1 + C=2*T=1 + T=2*V=0 <-> PB=1
6 C=1 + F=2 + C=0*F=1 + T=1*V=0 + T=2*V=0 <-> PB=1
7 C=1 + F=2 + C=0*T=2 + C=2*T=1 + T=2*V=0 <-> PB=1
8 C=1 + F=2 + C=0*T=2 + T=1*V=0 + T=2*V=0 <-> PB=1
```

Such commonalities can be reported as conclusive results.

Moreover, even though multiple solutions do not permit pinpointing the causal structure behind an outcome, they nonetheless allow for constraining the range of possibilities. In a context where the causes of some outcome are unknown it amounts to a significant gain of scientific insight when a study can show that the structure behind that outcome has one of a small number of possible forms, even if it cannot determine which one exactly.

However, the larger the amount of data-fitting solutions and the lower the amount of commonalities among them, the lower the overall informativeness of a CNA output. Indeed, if data fragmentation is high, meaning if there are many unobserved possible configurations, the ambiguity ratio in configurational causal modeling can reach dimensions where nothing at all can be concluded about the data-generating structure any more. Hence, a highly ambiguous result is on a par with a null result. A telling example of this sort is `d.volatile`, which was discussed in section 3.4 above (cf. also Baumgartner and Thiem 2017).

The model fit scores and solution attributes reported in the output objects of `cna()` and `csf()` often provide some leverage to narrow down the space of model candidates. For instance, if,

in a particular discovery context, there is reason to assume that data have been collected as exhaustively as possible, to the effect that most configurations compatible with an investigated causal structure should be contained in the data, the model space may be restricted to *csf* with a high score on exhaustiveness. By way of example, for `d.pban` a total of 14 *csf* are built at `cov = .95`:

```
R> ana.pban <- cna(d.pban, cov = .95, maxstep = c(6, 6, 10),
+     details = c("fa", "ex"))
R> csf.pban <- csf(ana.pban)
R> length(csf.pban$condition)

[1] 14
```

If only *csf* with `exhaustiveness >= .85` are considered, the amount of candidate *csf* is reduced to 2:

```
R> csf.pban.ex <- subset(csf.pban, exhaustiveness >= .85)
R> length(csf.pban.ex$condition)

[1] 2
```

To also resolve this final ambiguity, complexity may be brought to bear. Among equally data-fitting models, the less complex ones are generally preferable because they are less likely to be overfitted and make fewer causal claims, resulting in a lower error risk. In the above example, if complexity is required to be as low as possible, only one model remains:

```
R> subset(csf.pban.ex, complexity == min(csf.pban.ex$complexity))

  outcome condition                                con   cov complexity
1 PB=1    C=1 + F=2 + C=0*F=1 + C=2*V=0 <-> PB=1   1 0.952          6
  cyclic fm_empirical faithfulness exhaustiveness
1  FALSE         TRUE        0.941          0.889
Measures:
  con: standard consistency
  cov: standard coverage
```

Another route to ambiguity reduction is provided by the functional minimality constraint introduced in section 5.5. Unlike exhaustiveness, which evaluates *csf* by their relation to the data, functional minimality is a property of the *csf* themselves: it eliminates all solutions that account for an outcome on the basis of more factors than another solution in the CNA output. Applied to the analysis above, it reduces the 14 *csf* to 2:

```
R> csf.pban.fm <- csf(ana.pban, func.minimal.only = TRUE)
R> length(csf.pban.fm$condition)

[1] 2
```

```
R> csf.pban.fm[c("condition", "con", "cov")]

  condition                                          con   cov
1 C=1 + F=2 + C=0*F=1 + C=2*V=0 <-> PB=1             1 0.952
2 C=1 + F=2 + C=2*F=0 + C=0*F=1 + F=1*V=0 <-> PB=1   1 0.952
Measures:
  con: standard consistency
  cov: standard coverage
```

All 12 eliminated *csf* model PB=1 on the basis of the factors C, F, T and V, whereas the two retained ones make do with C, F and V while scoring the same consistency and coverage. What is more, the two retained *csf* are exactly the two that the exhaustiveness filter leaves above. Two routes drawing on entirely different information—a solution's fit to the data on the one hand, the factor sets of the solutions on the other—thus converge on the same pair of candidates, from which the complexity comparison above singles out the same *csf* again.

Clearly though, the fit parameters, functional minimality, and other solution attributes provided by `cna()` and `csf()` will not always facilitate complete ambiguity elimination. The evidence contained in data is often insufficient to draw determinate causal conclusions. In such instances, data-external sources of information, such as prior causal knowledge or background theories, may be available that can be used to suitably constrain the search space of `cna()` via the arguments `outcome`, `ordering`, or `exclude` (see section 3.3 above). This tends to bring down ambiguities significantly. Moreover, the next section will show how knowledge about individual cases in the data can be leveraged to select among the model candidates. Nevertheless, it may also be impossible to resolve all ambiguities when the evidence in the data is complemented by data-external sources of information.

The most important course of action in the face of ambiguities is to *render them transparent*. By default, readers of CNA publications should be informed about the degree of ambiguity. Full transparency with respect to model ambiguities, first, allows readers to determine for themselves how much confidence to have in the conclusions drawn in a study, and second, paves the way for follow-up studies that are purposefully designed to resolve previously encountered ambiguities.

6.4. "Back to the cases"

When CNA is applied to small- or intermediate-N data, researchers may be familiar with some or all of the cases in their data. For instance, they may know that in a particular case certain causes of an outcome are operative while others are not. Or they may know why certain cases are outliers or why others feature an outcome but none of the potential causes. A proper interpretation of a CNA result may therefore require that the performance of the obtained models be assessed on the case level and against the background of the available case knowledge.

The function that facilitates the evaluation of recovered *msc*, *asf*, and *csf* on the case level is `condition(x, ct, measures)`. Its first input is a character vector `x` specifying Boolean expressions, typically *asf* or *csf*. The second input is a data frame or configuration table `ct`. The `measures` argument is the same as in the `cna()` function (see section 3.2 above): it expects a character vector of length 2 indicating the measures to be used for evaluating

sufficiency and necessity. In case of *cs* or *mv* data, the output of `condition()` then highlights in which cases `x` is instantiated, whereas for *fs* data, the output lists relevant membership scores in exogenous and endogenous factors. Moreover, if `x` is an *asf* or *csf*, `condition()` issues their scores on the chosen `measures`.

To illustrate, we re-analyze `d.autonomy`:

```
R> dat.aut.2 <- d.autonomy[15:30, c("AU","EM","SP","CO","RE","DE")]
R> ana.aut.5 <- cna(dat.aut.2, outcome = c("EM","AU"), con = .91, cov = .91)
R> condition(csf(ana.aut.5)$condition, dat.aut.2)
```

That function call returns a list of three tables, each corresponding to one of the three *csf* contained in `ana.aut.5` and breaking the relevant *csf* down to the case level by contrasting the membership scores in the left-hand and right-hand sides of the component *asf*. A case with a higher left-hand score is one that pulls down the sufficiency measure (e.g. standard consistency), whereas a case with a higher right-hand score pulls down the necessity measure (e.g. standard coverage). For each *csf*, `condition()` moreover returns overall scores on the chosen evaluation `measures` as well as the corresponding scores for the component *asf*.

The three *csf* in `ana.aut.5` differ only with regard to their component *asf* for outcome *AU*. The function `group.by.outcome(condList)`, which takes an output object `condList` of `condition()` as input, lets us more specifically compare these different *asf* with respect to how they fare on the case level.

```
R> group.by.outcome(condition(asf(ana.aut.5)$condition, dat.aut.2))$AU
```

```
          SP EM*RE+re*DE EM*RE+CO*DE  AU | n.obs
ENacg1   1.0         1.0         1.0 1.0 |     1
ENacg2   0.6         0.4         0.4 0.4 |     1
ENacg3   0.8         0.6         0.9 0.8 |     1
ENacg4   0.6         1.0         1.0 1.0 |     1
ENacg5   0.4         0.4         0.4 0.4 |     1
ENacg6   0.6         0.7         0.7 0.6 |     1
ENacg7   1.0         0.8         0.8 1.0 |     1
ENacg8   1.0         1.0         1.0 1.0 |     1
ENacto1  0.4         0.4         0.6 0.4 |     1
ENacto2  0.4         0.4         0.4 0.4 |     1
ENacosa1 0.4         0.4         0.2 0.4 |     1
ENacosa2 0.4         0.4         0.4 0.2 |     1
ENacosa3 0.4         0.4         0.4 0.6 |     1
ENacat1  0.4         0.4         0.4 0.2 |     1
ENacat2  0.4         0.4         0.4 0.6 |     1
ENacat3  0.4         0.6         0.4 0.4 |     1
```

The first three columns of that table list the membership scores of each case in the left-hand sides of the *asf*, and the fourth column reports the membership scores in *AU*. The table shows that the first *asf* ($SP \leftrightarrow AU$) outperforms the other *asf* in cases ENacg3/6/7, ENacto1, ENacosa1, and ENacat3, while it is outperformed by another *asf* in cases ENacg2 and ENacg4. In all other cases, the three solution candidates fare equally. If the analyst is

closely familiar with some of these cases, performance differences on the case level can help to choose among the candidates. For instance, if it is known that there are no other factors operative in case ENacg7 than the ones contained in `dat.aut.2`, it follows that ENacg7's full membership in *AU* must be brought about by *SP*—which, in turn, disqualifies the other solutions. By contrast, if the absence of other relevant factors can be assumed for case ENacg4, the *asf* featuring *SP* as cause of *AU* is disqualified.

7. Benchmarking

Benchmarking the reliability of a method of causal inference is an essential element of method development and validation. In a nutshell, it amounts to testing to what degree the benchmarked method recovers the true data-generating structure Δ or proper substructures of Δ from data of varying quality. As Δ is not normally known in real-life discovery contexts, the reliability of a method cannot be assessed by applying it to real-life data. Instead, reliability benchmarking is done in so-called *inverse searches*, which reverse the order of causal discovery as it is commonly conducted in scientific practice. An inverse search comprises three steps:

(1) a data-generating causal structure Δ is presupposed/drawn (as ground truth),

(2) artificial data δ is simulated from Δ, possibly featuring various deficiencies (e.g. noise or fragmentation),

(3) δ is processed by the tested method in order to check whether its output meets the tested reliability benchmark.

A benchmark test can measure various properties of a method's output, for instance, whether it is error-free, correct or complete, etc. As real-life data are often fragmented, methods for MINUS discovery typically do not infer the complete Δ from a real-life δ but only proper substructures thereof (see section 2.4). Thus, since completeness is not CNA's primary aim, it should likewise not be the primary reliability benchmark for CNA; it is more important that its output scores high on *error-freeness* and *correctness*.

CNA's output, *viz.* the issued set $\mathbf{csf_O}$ of *csf*, is error-free iff it does not entail a causal claim that is false of the ground truth Δ (i.e. no false positive). That can be satisfied in two ways: either (i) $\mathbf{csf_O}$ is empty, meaning no causal inferences are drawn, or (ii) $\mathbf{csf_O}$ contains at least one[16] solution $\mathbf{m}_i$ that is correct of Δ, which is the case iff $\mathbf{m}_i$ is a submodel of Δ (for details on the submodel relation see section 2.4). So, $\mathbf{csf_O}$ satisfies the error-freeness benchmark iff it satisfies conditions (i) or (ii). With increasing stringency, $\mathbf{csf_O}$ can then be said to be correct of Δ iff condition (ii) is satisfied, meaning $\mathbf{csf_O}$ actually contains at least one solution $\mathbf{m}_i$ that is a submodel of Δ, and thus correct. Finally, completeness measures the informativeness of $\mathbf{csf_O}$, that is, the ratio of causal properties of Δ captured and revealed by the solutions in $\mathbf{csf_O}$.

The **cna** package provides many functionalities to conduct inverse searches that are tailor-made to benchmark the output of `cna()` and `csf()`. The functions `randomAsf()` and `randomCsf()` can be used to draw a data-generating structure Δ in step (1). `randomAsf(x)` generates a structure with a single outcome (i.e. a random *asf*) and `randomCsf(x)` an acyclic

[16]Recall from section 6.3 that an output containing multiple solutions is to be interpreted disjunctively; and a disjunction of solutions is true iff at least one solution is true.

multi-outcome structure (i.e. a random *csf*), where `x` is a data frame or `configTable` defining the factors and their possible values from which the structures are drawn. The function `selectCases()`, which has already been discussed in section 3.1.2, can be employed to simulate data δ in the course of step (2). Finally, `is.submodel(x, y)` determines whether models are related by the submodel relation, which, in turn, helps in assessing whether $\mathbf{csf_O}$ is true of Δ. `is.submodel()` takes a character vector `x` of *asf* as first input and tests whether the elements of that vector are submodels of `y`, which, in turn, is a character string of length 1 representing the target *asf* (i.e. Δ). If Δ is a *csf* with multiple outcomes, the function `causal_submodel(x, y)` from the **frscore** package should be used to determine whether `x` is true of `y`. Moreover, the function `identical.model(x, y)` is available to check whether `x` (which must have length 1) and `y` are identical.

Against that background, the following might be a core of an error-freeness benchmark test that simulates multi-value data with 20% missing observations and 10% random noise (i.e. cases incompatible with the ground truth), and that runs `cna()` and `csf()` using the evaluation `measures` of prevalence-adjusted consistency and antecedent-adjusted coverage at `con` = `cov` = 0.75 and giving the algorithm and `outcome` specification.

```
R> # Draw a ground truth with outcomes A=1 and B=2.
R> fullData <- allCombs(c(4,4,4,4,4)) |> ct2df()
R> groundTruth <- randomCsf(fullData, outcome = c("A=1", "B=2"), compl = 3)
R> # Generate ideal data for groundTruth.
R> idealData <- ct2df(selectCases(groundTruth, fullData))
R> # Introduce 20% fragmentation.
R> fragData <- idealData[-sample(1:nrow(idealData), nrow(idealData)*0.2), ]
R> # Add 10% random noise (cases incompatible with ground truth).
R> incompCases <- dplyr::setdiff(fullData, idealData)
R> x <- rbind(incompCases[sample(1:nrow(incompCases),
+     nrow(fragData) * 0.1), ], fragData)
R> # Run CNA with outcome specification and PAcon/AAcov as evaluation
R> # measures.
R> csfs <- csf(cna(x, outcome = c("A=1", "B=2"), con = .75, cov = .75,
+     maxstep = c(3, 3, 12), measures = c("PAcon", "AAcov")))
R> # Check whether no causal error (no false positive) is returned.
R> if(length(csfs$condition)==0) {
+     TRUE } else {any(unlist(lapply(csfs$condition,
+     function(x) frscore::causal_submodel(x, groundTruth, fullData))))}
```

Every re-run of this code chunk generates a different ground truth and different data. In some runs CNA passes the test, in others it does not. To determine CNA's error-freeness ratio under these test conditions, the above core must be embedded in a suitable test loop. To estimate CNA's overall error-freeness ratio, the test conditions should be systematically varied by, for instance, varying the complexity of the ground truth, the degree of fragmentation and noise, the evaluation measures and corresponding thresholds, or by drawing the noise with a bias or supplying CNA with more or less prior causal information via `ordering` or `exclude`. Correctness and completeness tests can be designed analogously, by suitably modifying the last line that evaluates the solution object `csfs`. For single-outcome structures (*asf*), benchmark tests with some of the above variations have been conducted in (Baum-

gartner and Ambühl 2020; Baumgartner and Falk 2023b; Swiatczak and Baumgartner 2024; De Souter 2024; De Souter and Baumgartner 2025); corresponding tests for multi-outcome structures (*csf*) have been carried out in (Parkkinen and Baumgartner 2023).

8. Summary

This vignette introduced the theoretical foundations as well as the main functions of the **cna** R package for configurational causal inference and modeling with Coincidence Analysis (CNA). Moreover, we explained how to interpret the output of CNA, provided some guidance for how to use various model fit parameters for the purpose of ambiguity reduction, and supplied a benchmarking template.

CNA is currently the only method searching for (M)INUS causation in data that builds multi-outcome models and, hence, not only orders causes conjunctively and disjunctively but also sequentially. Moreover, it builds causal models on the basis of a bottom-up algorithm that is unique among configurational comparative methods and gives CNA an edge over other methods in guaranteeing the redundancy-freeness of its models, which, in turn, is crucial for their causal interpretability. Overall, CNA constitutes a powerful methodological alternative for researchers interested in causal structures featuring conjunctivity and disjunctivity. The **cna** package makes that inferential power available to end-users.

Acknowledgments

We are grateful to Alrik Thiem, Martyna Klein, Jonathan Freitas, and Luna De Souter for helpful comments on earlier drafts of this vignette, and we thank the Toppforsk-program of the Trond Mohn Foundation and the University of Bergen (grant nr. 811886), the Research Council of Norway (grant nr. 326215), and the Swiss National Science Foundation (grant nr. PP00P1_144736/1) for generous support of the research behind the **cna** package over the years.

References

Ambühl M, Baumgartner M (2025). ***cnaOpt**: Optimizing Consistency and Coverage in Configurational Causal Modeling*. R Package Version 0.5.3. URL https://cran.r-project.org/package=cnaOpt.

Baumgartner M (2009a). "Inferring Causal Complexity." *Sociological Methods & Research*, **38**, 71–101.

Baumgartner M (2009b). "Uncovering Deterministic Causal Structures: A Boolean Approach." *Synthese*, **170**, 71–96.

Baumgartner M (2013). "A Regularity Theoretic Approach to Actual Causation." *Erkenntnis*, **78**, 85–109.

Baumgartner M (2015). "Parsimony and Causality." *Quality & Quantity*, **49**, 839–856.

Baumgartner M (2020). "Causation." In D Berg-Schlosser, B Badie, L Morlino (eds.), *The SAGE Handbook of Political Science*, pp. 305–321. SAGE, London.

Baumgartner M, Ambühl M (2020). "Causal Modeling with Multi-Value and Fuzzy-Set Coincidence Analysis." *Political Science Research and Methods*, **8**, 526–542. doi:10.1017/psrm.2018.45.

Baumgartner M, Ambühl M (2021). "Optimizing Consistency and Coverage in Configurational Causal Modeling." *Sociological Methods & Research*, **52**(3), 1288–1320. doi:10.1177/0049124121995554.

Baumgartner M, Falk C (2023a). "Boolean Difference-Making: A Modern Regularity Theory of Causation." *The British Journal for the Philosophy of Science*, **74**(1), 171–197. doi:10.1093/bjps/axz047.

Baumgartner M, Falk C (2023b). "Configurational Causal Modeling and Logic Regression." *Multivariate Behavioral Research*, **58**(2), 292–310. doi:10.1080/00273171.2021.1971510.

Baumgartner M, Falk C (forthcoming). "Introducing Causal Hypergraphs." *Quality & Quantity*. URL https://m-baum.github.io/docs/causal_hypergraphs.pdf.

Baumgartner M, Thiem A (2017). "Model Ambiguities in Configurational Comparative Research." *Sociological Methods & Research*, **46**(4), 954–987.

Baumgartner M, Thiem A (2020). "Often Trusted but Never (Properly) Tested: Evaluating Qualitative Comparative Analysis." *Sociological Methods & Research*, **49**, 279–311. doi:10.1177/0049124117701487.

Beirlaen M, Leuridan B, Van De Putte F (2018). "A logic for the discovery of deterministic causal regularities." *Synthese*, **195**(1), 367–399. doi:10.1007/s11229-016-1222-x.

Bowran AP (1965). *A Boolean Algebra. Abstract and Concrete*. Macmillan, London.

Brambor T, Clark WR, Golder M (2006). "Understanding Interaction Models: Improving Empirical Analyses." *Political Analysis*, **14**(1), 63–82. doi:10.1093/pan/mpi014.

Braumoeller B (2015). ***QCAfalsePositive**: Tests for Type I Error in Qualitative Comparative Analysis (QCA)*. R package version 1.1.1, URL https://CRAN.R-project.org/package=QCAfalsePositive.

Bretto A (2013). *Hypergraph Theory: An Introduction*. Springer International Publishing.

Cronqvist L, Berg-Schlosser D (2009). "Multi-Value QCA (mvQCA)." In B Rihoux, CC Ragin (eds.), *Configurational Comparative Methods: Qualitative Comparative Analysis (QCA) and Related Techniques*, pp. 69–86. Sage Publications, London.

Csikszentmihalyi M (1975). *Beyond Boredom and Anxiety*. Jossey-Bass Publishers, San Francisco.

Culverhouse R, Suarez BK, Lin J, Reich T (2002). "A Perspective on Epistasis: Limits of Models Displaying No Main Effect." *The American Journal of Human Genetics*, **70**(2), 461–471. doi:10.1086/338759.

De Souter L (2024). "Evaluating Boolean Relationships in Configurational Comparative Methods." *Journal of Causal Inference*, **12**(1). doi:10.1515/jci-2023-0014.

De Souter L, Baumgartner M (2025). "New sufficiency and necessity measures for model building with Coincidence Analysis." *Zenodo*. URL https://doi.org/10.5281/zenodo.13619580.

Dusa A (2026). ***QCA**: A Package for Qualitative Comparative Analysis*. R Package Version 3.25. URL https://cran.r-project.org/package=QCA.

Eberhardt F (2013). "Experimental Indistinguishability of Causal Structures." *Philosophy of Science*, **80**(5), 684–696.

Falk C, Ambühl M, Baumgartner M (2024). ***causalHyperGraph**: Drawing Causal Hypergraphs*. R Package Version 0.1.0. URL https://cran.r-project.org/package=causalHyperGraph.

Gallo G, Longo G, Pallottino S, Nguyen S (1993). "Directed Hypergraphs and Applications." *Discrete Applied Mathematics*, **42**(2), 177–201. doi:10.1016/0166-218X(93)90045-P.

Gil-Pons R, Ward M, Miller L (2024). "Finding (s,d)-Hypernetworks in F-Hypergraphs is NP-Hard." *Information Processing Letters*, **184**, 106433. doi:10.1016/j.ipl.2023.106433.

Graßhoff G, May M (2001). "Causal Regularities." In W Spohn, M Ledwig, M Esfeld (eds.), *Current Issues in Causation*, pp. 85–114. Mentis, Paderborn.

Greenland S, Pear J, Robins JM (1999). "Causal Diagrams for Epidemiologic Research." *Epidemiology*, **10**(1), 37–48.

Hájek P (1998). *Metamathematics of Fuzzy Logic*. Kluwer, Dordrecht.

Hume D (1999 (1748)). *An Enquiry Concerning Human Understanding*. Oxford University Press, Oxford.

Kalisch M, Maechler M, Colombo D, Maathuis MH, Buehlmann P (2012). "Causal Inference Using Graphical Models with the R Package **pcalg**." *Journal of Statistical Software*, **47**(11), 1–26.

Kooperberg C, Ruczinski I (2005). "Identifying Interacting SNPs Using Monte Carlo Logic Regression." *Genetic Epidemiology*, **28**(2), 157–170. doi:10.1002/gepi.20042.

Kooperberg C, Ruczinski I (2023). ***LogicReg**: Logic Regression*. R package version 1.6.6. URL https://CRAN.R-project.org/package=LogicReg.

Lemmon EJ (1965). *Beginning Logic*. Chapman & Hall, London.

Mackie JL (1974). *The Cement of the Universe. A Study of Causation*. Clarendon Press, Oxford.

Oana IE, Medzihorsky J, Quaranta M, Schneider CQ (2025). ***SetMethods**: Functions for Set-Theoretic Multi-Method Research and Advanced QCA*. R package version 4.1, URL https://CRAN.R-project.org/package=SetMethods.

Parkkinen VP, Baumgartner M (2023). "Robustness and Model Selection in Configurational Causal Modeling." *Sociological Methods & Research*, **52**(1), 176–208.

Parkkinen VP, Baumgartner M (2025). ***frscore**: Functions for Calculating Fit-Robustness of CNA-solutions*. R Package Version 0.5.2. URL https://CRAN.R-project.org/package=frscore.

Ragin CC (2006). "Set Relations in Social Research: Evaluating Their Consistency and Coverage." *Political Analysis*, **14**(3), 291–310.

Ragin CC (2008). *Redesigning Social Inquiry: Fuzzy Sets and Beyond*. University of Chicago Press, Chicago.

Rihoux B, Ragin CC (eds.) (2009). *Configurational Comparative Methods. Qualitative Comparative Analysis (QCA) and Related Techniques*. Sage, Thousand Oaks.

Ruczinski I, Kooperberg C, LeBlanc M (2003). "Logic Regression." *Journal of Computational and Graphical Statistics*, **12**(3), 475–511. doi:10.1198/1061860032238.

Schneider CQ, Wagemann C (2012). *Set-Theoretic Methods: A User's Guide for Qualitative Comparative Analysis (QCA) and Fuzzy-Sets in the Social Sciences*. Cambridge University Press, Cambridge.

Schwender H, Tietz T (2024). ***logicFS**: Identification of SNP Interactions*. R package version 2.26.0. doi:10.18129/B9.bioc.logicFS.

Siblini W, Fréry J, He-Guelton L, Oblé F, Wang YQ (2020). "Master Your Metrics with Calibration." In MR Berthold, A Feelders, G Krempl (eds.), *Advances in Intelligent Data Analysis XVIII*, pp. 457–469. Springer, Cham. doi:10.1007/978-3-030-44584-3_36.

Simon HA (1954). "Spurious Correlation: A Causal Interpretation." *Journal of the American Statistical Association*, **49**(267), 467–479.

Spirtes P, Glymour C, Scheines R (2000). *Causation, Prediction, and Search*. 2 edition. MIT Press, Cambridge.

Swiatczak MD (2021). "Different Algorithms, Different Models." *Quality & Quantity*, **56**(4), 1913–1937. doi:10.1007/s11135-021-01193-9.

Swiatczak MD, Baumgartner M (2024). "Data Imbalances in Coincidence Analysis: A Simulation Study." *Sociological Methods & Research*. ISSN 1552-8294. doi:10.1177/00491241241227039.

Thiem A (2018). ***QCApro**: Advanced Functionality for Performing and Evaluating Qualitative Comparative Analysis*. R Package Version 1.1-2. URL https://CRAN.R-project.org/package=QCApro.

Thiem A, Duşa A (2013). *Qualitative Comparative Analysis With R: A User's Guide*. Springer, New York, NY.

Whitaker RG, Sperber N, Birken S, Baumgartner M, Thiem A, Cragun D, Damschroder L, Miech E, Slade A (2020). "Coincidence Analysis: A New Method for Causal Inference in Implementation Science." *Implementation Science*, **15**. doi:10.1186/s13012-020-01070-3.

Yakovchenko V, Miech EJ, Chinman MJ, Chartier M, Gonzalez R, Kirchner JE, Morgan TR, Park A, Powell BJ, Proctor EK, Ross D, Waltz TJ, Rogal SS (2020). "Strategy Configurations Directly Linked to Higher Hepatitis C Virus Treatment Starts: An Applied Use of Configurational Comparative Methods." *Medical Care*, **58**(5). doi:10.1097/MLR.0000000000001319.

Zhang J, Zhang K (2023). "A New Minimality Condition for Boolean Accounts of Causal Regularities." *Erkenntnis*, **90**(1), 67–86. doi:10.1007/s10670-023-00685-4.

Appendix

Partial structural redundancies

As discussed in section 2.3, conjunctively concatenating atomic MINUS-formulas or *asf* may give rise to structural redundancies (cf. also Baumgartner and Falk 2023a), which are eliminated in stage 4 of the CNA algorithm. While structural redundancies of whole *asf* can occur in both ideal and noisy data, the latter type of data may induce yet another, but related, type of redundancy, which is not documented anywhere in the research literature and, thus, requires explicit discussion in this Appendix.

When data do not feature strict Boolean dependencies, building *csf* from the inventory of *asf* recovered in stage 3 of the CNA algorithm may lead to the redundancy of proper parts of *asf*—parts which are not redundant when those *asf* are considered in isolation. That is, a complex structure can entail that one of its *asf* has a redundant proper part, which redundancy, however, is not visible in the data. We call this a *partial structural redundancy*.

A concrete example helps to clarify the problem. Consider the solutions obtained from analyzing the data set `d.autonomy` without elimination of partial structural redundancies. This is achieved by setting the developer argument `inus.only` to `FALSE`, which adds an additional column, `inus`, to the output, indicating with `TRUE`/`FALSE` whether the formula in the corresponding row qualifies as a MINUS-formula:

```
R> printCols <- c("condition", "con", "cov", "inus")
R> csf(cna(d.autonomy, ordering = "AU", con = .9, cov = .94,
+     maxstep = c(2, 2, 8), inus.only = FALSE))[printCols]
```

```
  condition                                      con   cov  inus
1 (SP*RE + CO*cn <-> EM)*(ci + EM*co <-> SP) 0.938 0.947  TRUE
2 (SP*co + CO*cn <-> EM)*(ci + EM*co <-> SP) 0.938 0.947  TRUE
3 (SP*RE + ci*cn <-> EM)*(ci + EM*co <-> SP) 0.938 0.947 FALSE
4 (SP*RE + CO*cn <-> EM)*(ci + EM*RE <-> SP) 0.928 0.947  TRUE
5 (SP*co + CO*cn <-> EM)*(ci + EM*RE <-> SP) 0.928 0.947  TRUE
6 (SP*RE + ci*cn <-> EM)*(ci + EM*RE <-> SP) 0.928 0.947 FALSE
Measures:
  con: standard consistency
  cov: standard coverage
```

Solution #3 in that list logically entails (6):

$$(SP{*}RE + ci{*}cn \leftrightarrow EM){*}(ci + EM \leftrightarrow SP) \tag{6}$$

That is, if the behavior of EM is regulated by the first *asf* in solution #3 and (6), *co* in #3—for pure logical reasons—cannot make a difference to SP and, hence, is redundant. That partial structural redundancy, however, is not visible in the data `d.autonomy` where EM alone (i.e. without *co*) is not sufficient for SP with standard consistency 0.9, which is the threshold chosen for the above analysis. By itself, EM only reaches a standard consistency of 0.891 for SP, which can be shown using the `condTbl()` function as follows:

```
R> condTbl("EM -> SP", configTable(d.autonomy))
```

```
  outcome condition   con   cov complexity
1 SP       EM -> SP 0.891 0.863          1
Measures:
  con: standard consistency
  cov: standard coverage
```

Hence, the data suggest that *co* makes a difference to SP, to the effect that meeting `con` = 0.9 for all *msc* requires $EM{*}co$ (and not EM alone) to be treated as cause of SP. At the same time, that (6) logically follows from solution #3 implies that it is logically excluded that *co* is a difference-maker of SP in the context of solution #3. The result is a contradiction: the data call for including *co* as cause of SP, whereas the structure inferred from that data entails that *co* not be included.

The case of solution #6 is analogous. Solution #6 not only entails (6) but is logically equivalent to it. The upshot is the same: the data determine that some causal relevance relation obtains, which is logically excluded by the very structure inferred from the data. Such inconsistencies cannot be resolved by modifying solutions #3 and #6; rather, these solutions are not, and cannot be transformed into, well-formed MINUS-formulas that would meet `con`. They must be eliminated from the output. This is exactly what happens when `cna()` and `csf()` run normally, that is, without setting `inus.only` to `FALSE`:

```
R> csf(cna(d.autonomy, ordering = "AU", con = .9, cov = .94,
+     maxstep = c(2, 2, 8), details = "inus"))[printCols]
```

```
  condition                                    con   cov inus
1 (SP*RE + CO*cn <-> EM)*(ci + EM*co <-> SP) 0.938 0.947 TRUE
2 (SP*co + CO*cn <-> EM)*(ci + EM*co <-> SP) 0.938 0.947 TRUE
3 (SP*RE + CO*cn <-> EM)*(ci + EM*RE <-> SP) 0.928 0.947 TRUE
4 (SP*co + CO*cn <-> EM)*(ci + EM*RE <-> SP) 0.928 0.947 TRUE
Measures:
  con: standard consistency
  cov: standard coverage
```

Affiliation:

Michael Baumgartner
University of Bergen
Department of Philosophy
Postboks 7805
5020 Bergen
Norway
E-mail: michael.baumgartner@uib.no
URL: https://m-baum.github.io

Mathias Ambühl
Consult AG Statistical Services
Tramstrasse 10
8050 Zürich
E-mail: mathias.ambuehl@consultag.ch